



H-모빌리티 클래스

로보틱스 직무 심화 교육

공통

정문호

로봇학부
광운대학교



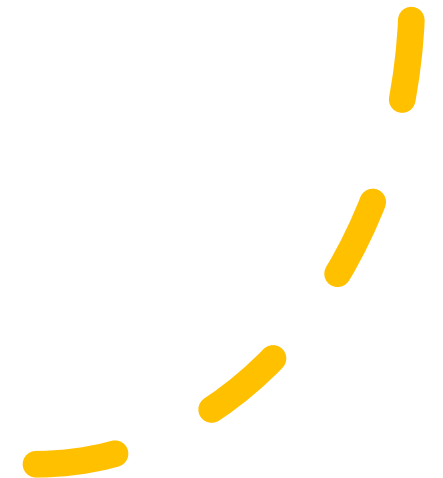
로봇 비전 기초

레슨1. Camera Calibration

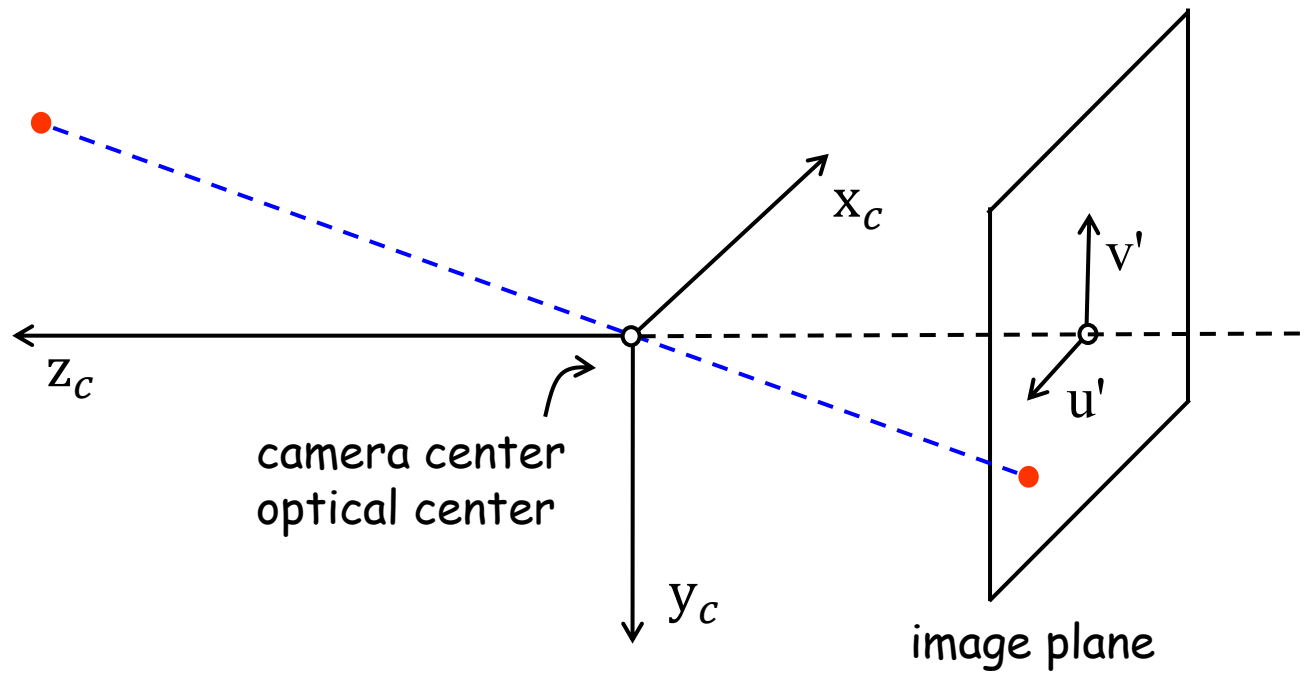
레슨2. Robot-Eye Calibration

Camera Calibration

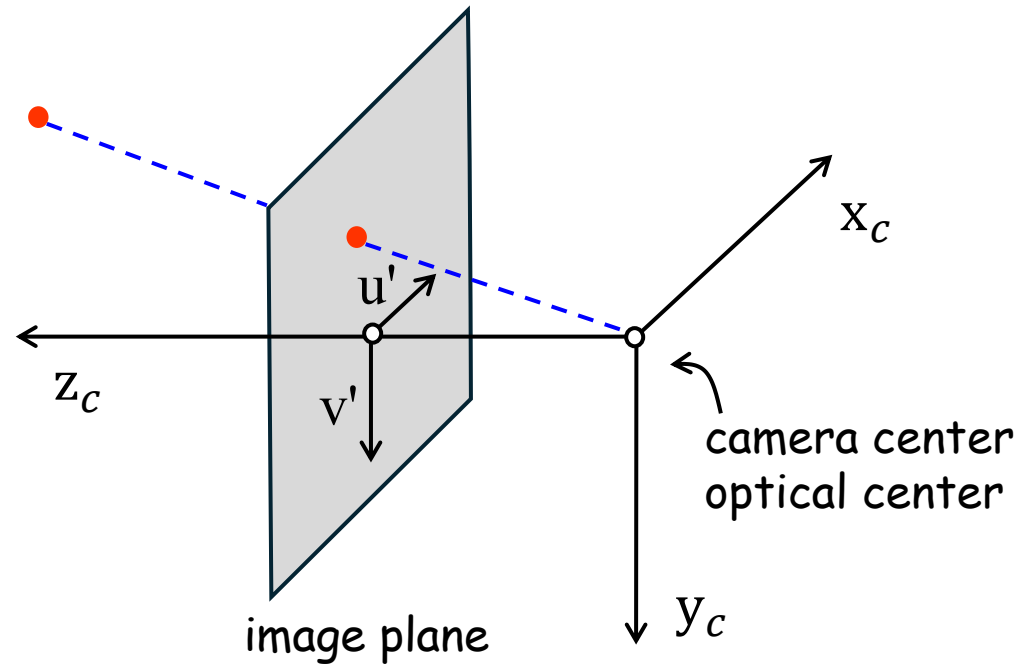
- Camera Geometry
- Zhang's Method
- Fine-Tuning of Intrinsic Parameters



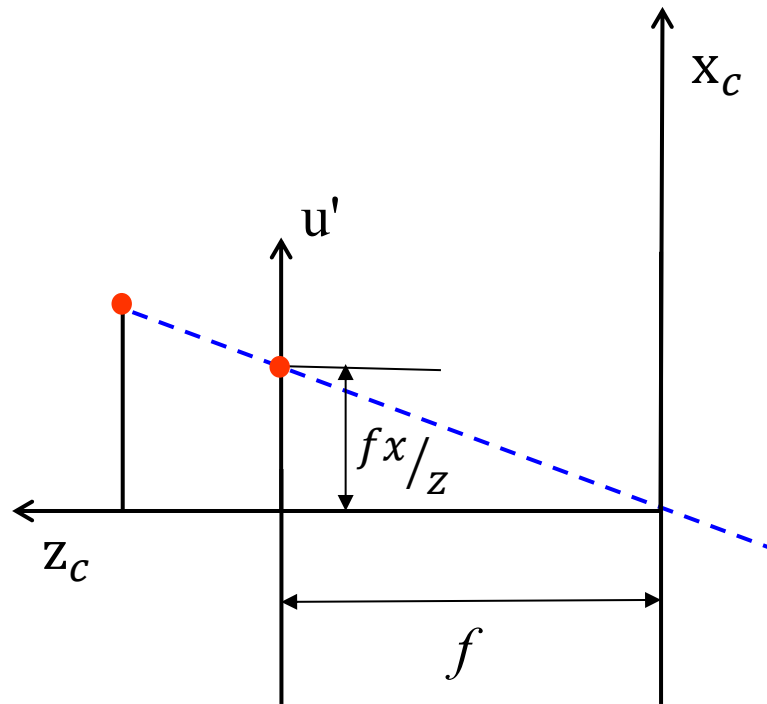
□ Camera Geometry



□ Camera Geometry



□ Perspective Projection



From the two similar triangles

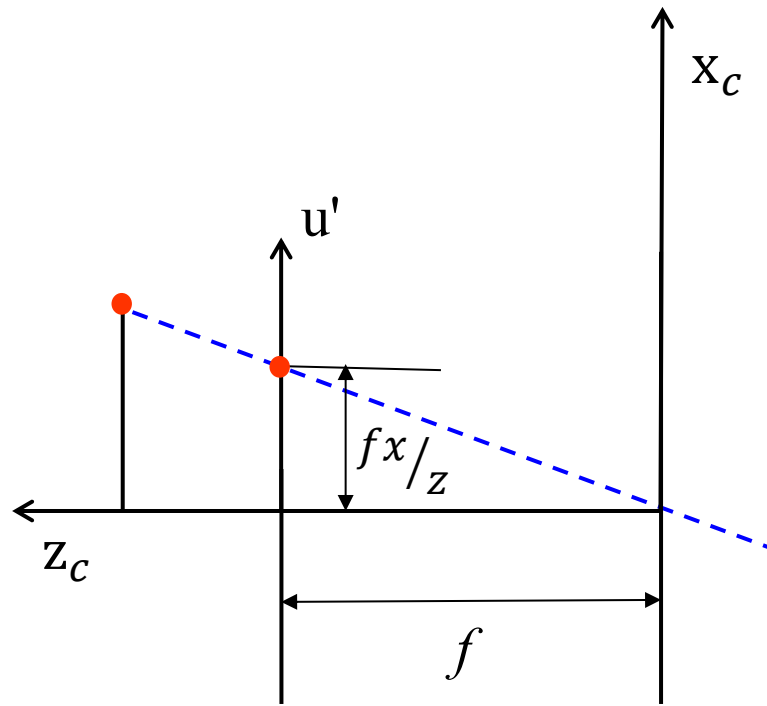
$$z : x = f : u'$$

$$u' = \frac{fx}{z}$$

Similarly,

$$v' = \frac{fy}{z}$$

□ Perspective Projection



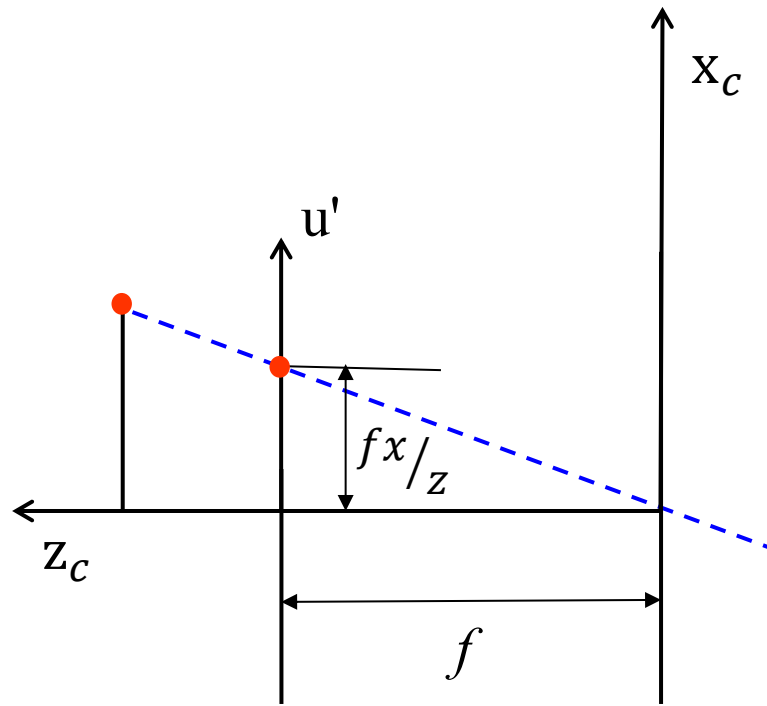
$$(x, y, z)^T \mapsto (u', v') = \left(\frac{fx}{z}, \frac{fy}{z} \right)$$

$$\begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \mapsto \begin{pmatrix} U' \\ V' \\ S' \end{pmatrix} = \begin{pmatrix} fx \\ fy \\ z \end{pmatrix} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$u' = \frac{U'}{S'}, \quad v' = \frac{V'}{S'}, \quad z = S'$$

linear projection in homogeneous coordinates !

□ Perspective Projection



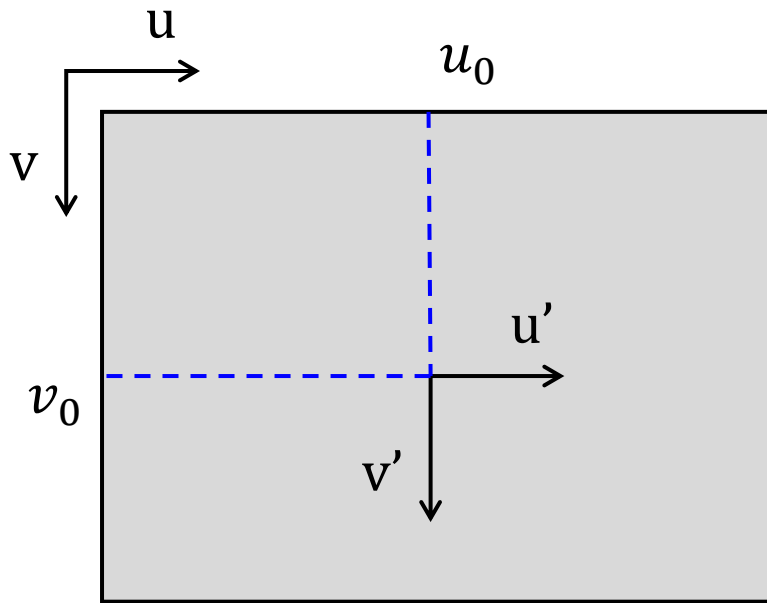
$$(x, y, z)^T \mapsto (u', v') = \left(\frac{fx}{z}, \frac{fy}{z} \right)$$

$$\begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \mapsto \begin{pmatrix} U' \\ V' \\ S' \end{pmatrix} = \begin{pmatrix} fx \\ fy \\ z \end{pmatrix} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$\tilde{\mathbf{U}} = \mathbf{P} \tilde{\mathbf{X}}_c$$

$$\mathbf{P} = \text{diag}(f, f, 1) [\mathbf{I} \mid \mathbf{0}]$$

□ Principal Point



(u_0, v_0) : principal point
image center

$$(x, y, z)^T \mapsto (u', v') \mapsto (u, v)$$

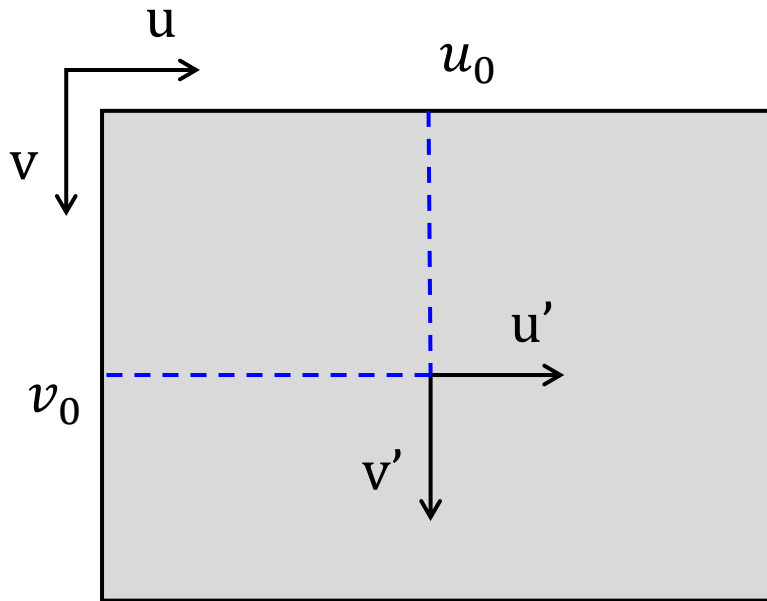
$$(u, v) = (u' + u_0, v' + v_0)$$

$$= (U'/S' + u_0, V'/S' + v_0)$$

$$\begin{pmatrix} U \\ V \\ S \end{pmatrix} = \begin{pmatrix} uS \\ vS \\ S \end{pmatrix} = \begin{pmatrix} (u' + u_0)S \\ (v' + v_0)S \\ S \end{pmatrix} \quad \because S' = S$$

$$= \begin{bmatrix} f & 0 & u_0 \\ 0 & f & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

□ Principal Point



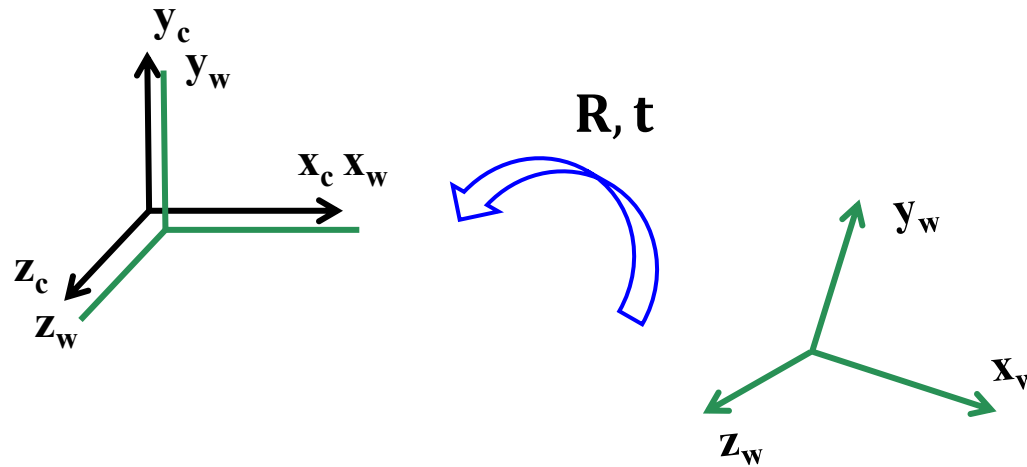
$$\begin{pmatrix} U \\ V \\ S \end{pmatrix} = \begin{bmatrix} f & 0 & u_0 \\ 0 & f & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$\tilde{\mathbf{U}} = \mathbf{K} [\mathbf{I} \mid \mathbf{0}] \tilde{\mathbf{X}}_c$$

$$\mathbf{K} = \begin{bmatrix} f & 0 & u_0 \\ 0 & f & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

calibration matrix
camera matrix
intrinsic parameters

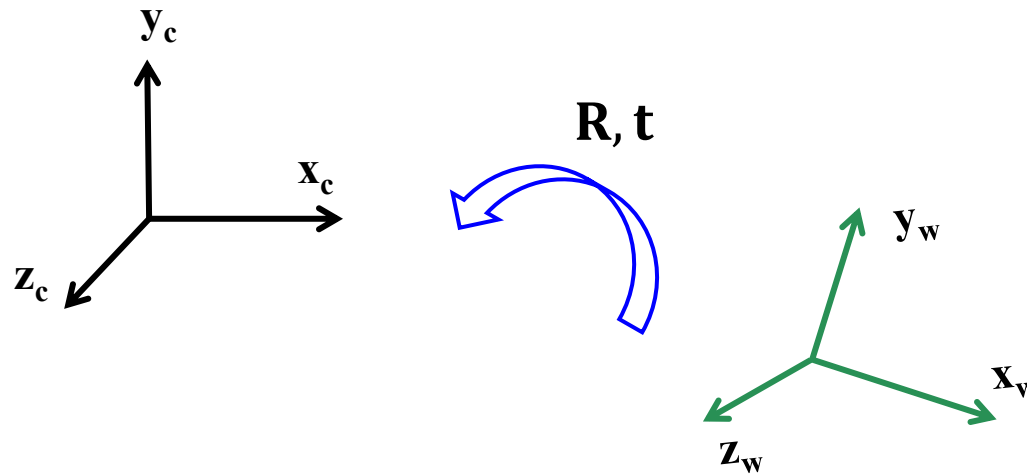
□ Rotation and Translation



$$\mathbf{x}_c = \mathbf{R} \begin{pmatrix} x_w \\ y_w \\ z_w \end{pmatrix} + \mathbf{t}$$

$\mathbf{R}, \mathbf{t}$: transform from the world coordinates to the camera coordinates - $\mathbf{R}_w^c, \mathbf{t}_w^c$

□ Rotation and Translation



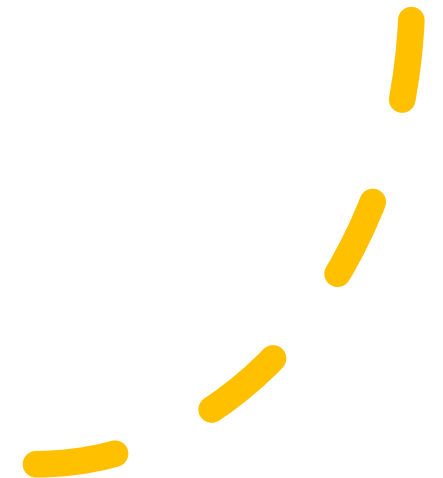
$$\tilde{\mathbf{X}}_c = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0 & 1 \end{bmatrix} \begin{pmatrix} x_w \\ y_w \\ z_w \\ 1 \end{pmatrix} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0 & 1 \end{bmatrix} \tilde{\mathbf{X}}_w$$

Substituting this into $\tilde{\mathbf{U}} = \mathbf{K} [\mathbf{I} | \mathbf{0}] \tilde{\mathbf{X}}_c$ gives $\tilde{\mathbf{U}} = \mathbf{K} [\mathbf{R} | \mathbf{t}] \tilde{\mathbf{X}}_w$.

$$\therefore \tilde{\mathbf{U}} = \mathbf{P} \tilde{\mathbf{X}}_w \text{ where } \mathbf{P} = \mathbf{K} [\mathbf{R} | \mathbf{t}]$$

Camera Calibration

- Camera Geometry
- Zhang's Method
- Fine-Tuning of Intrinsic Parameters



□ Camera Calibration

$$\tilde{\mathbf{U}} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \tilde{\mathbf{X}}_w \quad \mathbf{K} = \begin{bmatrix} f & 0 & u_0 \\ 0 & f & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

$[m], [mm]$
↓
 $[pixel]$

pixel/mm

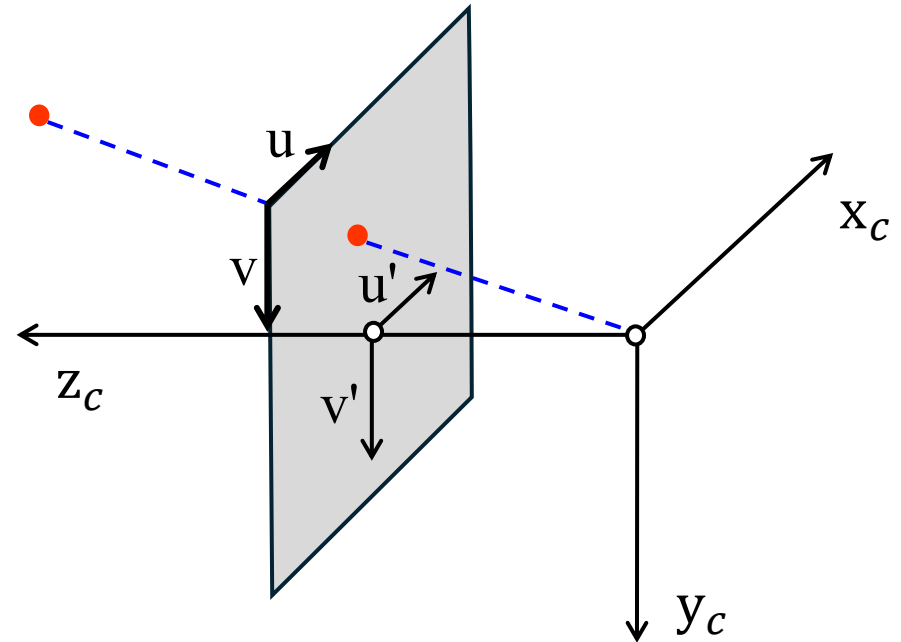
$$\mathbf{K} = \begin{bmatrix} k_u & 0 & 0 \\ 0 & k_v & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} f & 0 & u_0 \\ 0 & f & v_0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \alpha_u & 0 & k_u u_0 \\ 0 & \alpha_v & k_v v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\equiv \begin{bmatrix} \alpha & 0 & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

focal length in
pixel unit

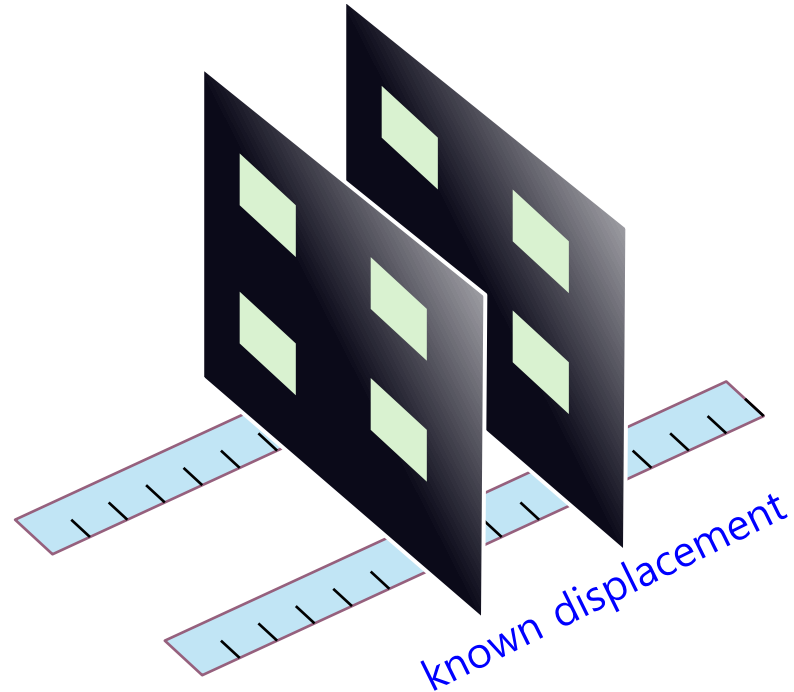
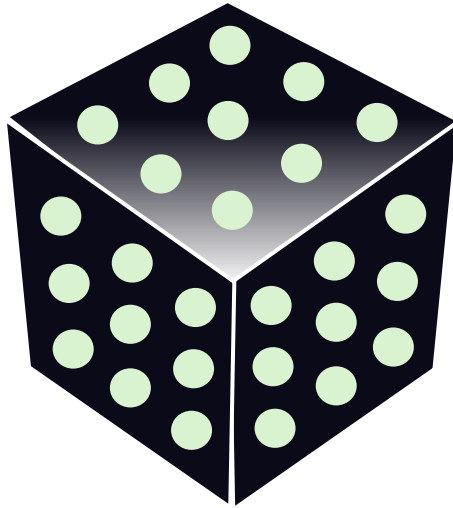
$$\mathbf{P} = \mathbf{K} \underbrace{\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}}_{\text{extrinsic parameters}}$$

intrinsic parameters



❑ Classical Approaches

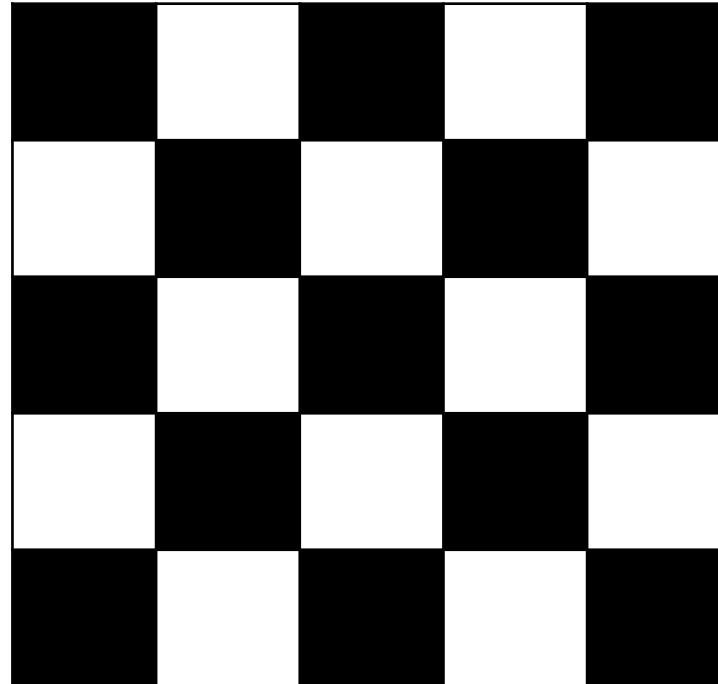
- Use precisely known 3D points



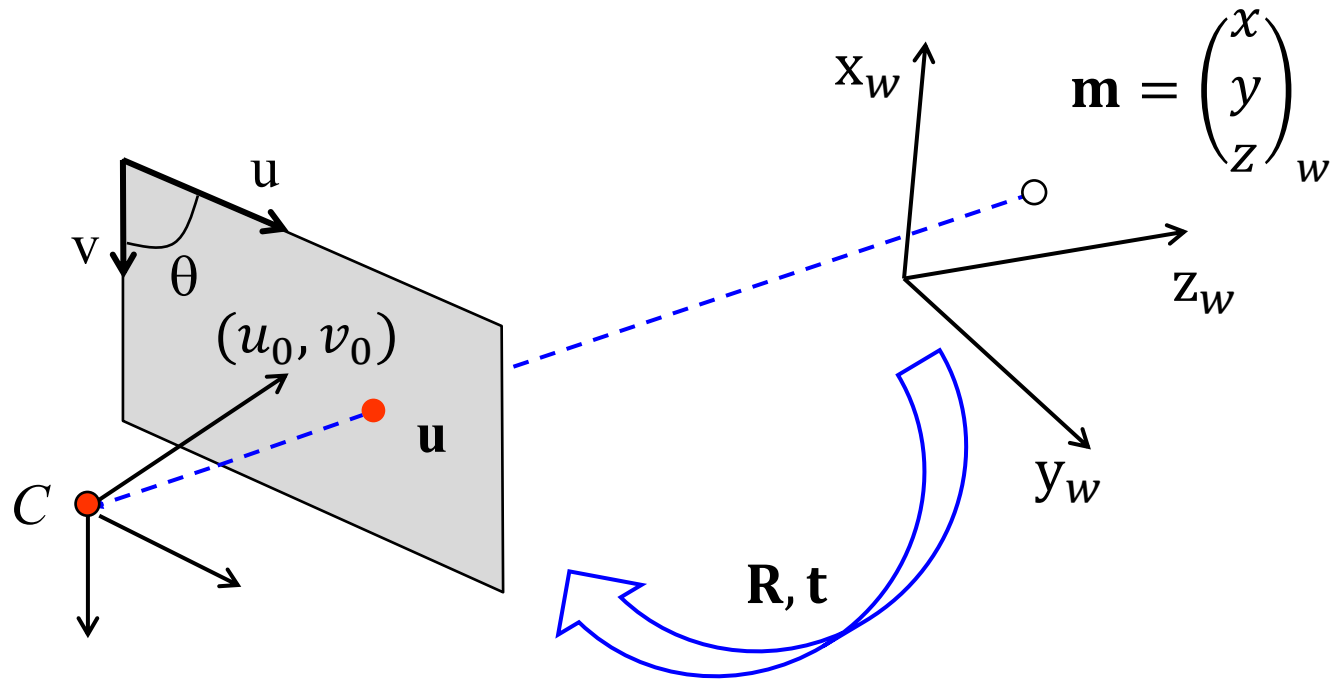
- Shortcomings
 - very expensive to make such a calibration target
 - not flexible

□ Zhang's Method

- Use only one plane
 - Print a pattern on a paper
 - Attach the paper on a planar surface
 - Show the plane freely a few times to the camera
- Advantages
 - Cheap
 - Flexible
 - Robust



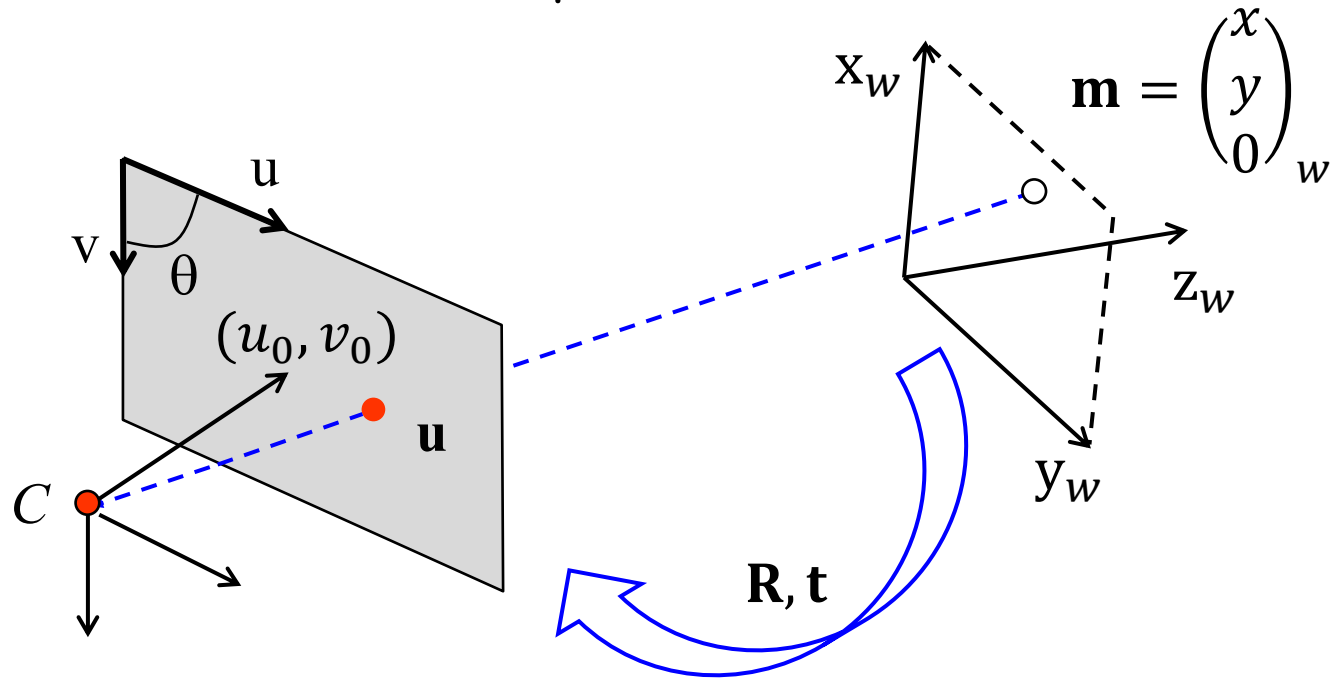
□ Perspective Projection



$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} \alpha & \gamma & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix}}_{\mathbf{K}} \underbrace{\begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{r}_3 & \mathbf{t} \end{bmatrix}}_{[\mathbf{R} \quad \mathbf{t}]} \underbrace{\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}}_{\tilde{\mathbf{M}}}$$

□ Plane projection

- Assuming that $z = 0$ for a model point,



$$\tilde{\mathbf{U}} = \mathbf{H}\tilde{\mathbf{M}} \text{ where } \mathbf{H} = \mathbf{K} [\mathbf{r}_1 \quad \mathbf{r}_2 \quad \mathbf{t}], \tilde{\mathbf{U}} = \begin{pmatrix} u \\ v \\ 1 \end{pmatrix}, \tilde{\mathbf{M}} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

□ How to Estimate Homography

$$\tilde{\mathbf{U}} = \mathbf{H}\tilde{\mathbf{M}}$$

defined up to scale

$$\begin{pmatrix} su \\ sv \\ s \end{pmatrix} = \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

1. Set up the linear equation: $\|\mathbf{A}\mathbf{h}\| = 0$, where $\mathbf{h} = (a \ b \ c \ \dots \ i)^T$, $\|\mathbf{h}\| = 1$

$$\begin{aligned} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \times \mathbf{H}\tilde{\mathbf{M}} &= \text{skew} \left(\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \right) \begin{pmatrix} \mathbf{h}_1^T \tilde{\mathbf{M}} \\ \mathbf{h}_2^T \tilde{\mathbf{M}} \\ \mathbf{h}_3^T \tilde{\mathbf{M}} \end{pmatrix} = \begin{pmatrix} 0 & -1 & v \\ 1 & 0 & -u \\ -v & u & 0 \end{pmatrix} \begin{pmatrix} \mathbf{h}_1^T \tilde{\mathbf{M}} \\ \mathbf{h}_2^T \tilde{\mathbf{M}} \\ \mathbf{h}_3^T \tilde{\mathbf{M}} \end{pmatrix} = \begin{pmatrix} v\mathbf{h}_3^T \tilde{\mathbf{M}} - \mathbf{h}_2^T \tilde{\mathbf{M}} \\ \mathbf{h}_1^T \tilde{\mathbf{M}} - u\mathbf{h}_3^T \tilde{\mathbf{M}} \\ u\mathbf{h}_2^T \tilde{\mathbf{M}} - v\mathbf{h}_1^T \tilde{\mathbf{M}} \end{pmatrix} \\ &= \begin{pmatrix} 0^T & -\tilde{\mathbf{M}}^T & v\tilde{\mathbf{M}}^T \\ \tilde{\mathbf{M}}^T & 0^T & -u\tilde{\mathbf{M}}^T \\ -v\tilde{\mathbf{M}}^T & u\tilde{\mathbf{M}}^T & 0^T \end{pmatrix} \begin{pmatrix} \mathbf{h}_1 \\ \mathbf{h}_2 \\ \mathbf{h}_3 \end{pmatrix} = 0. \end{aligned}$$

□ How to Estimate Homography

Letting $\mathbf{A}_k = \begin{pmatrix} 0^T & -\tilde{\mathbf{M}}^T & v\tilde{\mathbf{M}}^T \\ \tilde{\mathbf{M}}^T & 0^T & -u\tilde{\mathbf{M}}^T \\ -v\tilde{\mathbf{M}}^T & u\tilde{\mathbf{M}}^T & 0^T \end{pmatrix}_k$ where $k = 1, \dots, N$

gives, $\|\mathbf{A}\mathbf{h}\| = 0$ where $\mathbf{A} = \begin{pmatrix} \mathbf{A}_1 \\ \vdots \\ \mathbf{A}_N \end{pmatrix}$

2. $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ (← Singular Value Decomposition)

3. $\mathbf{h}$ is the last column of $\mathbf{V}$

- ① Let $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T$, where $\mathbf{U}$ is $m \times n$ orthonormal, $\mathbf{S}$ is $n \times n$ diagonal with descending order, and $\mathbf{V}^T$ is $n \times n$ orthonormal.
- ② From orthonormality of $\mathbf{U}$, $\mathbf{V}$ follows that $\|\mathbf{U}\mathbf{S}\mathbf{V}^T\mathbf{h}\| = \|\mathbf{S}\mathbf{V}^T\mathbf{h}\|$ and $\|\mathbf{V}^T\mathbf{h}\| = \|\mathbf{h}\|$.
- ③ Substitute $\mathbf{y} = \mathbf{V}^T\mathbf{h}$. Now, we minimize $\|\mathbf{S}\mathbf{y}\|$ subject to $\|\mathbf{y}\| = 1$.
- ④ Since $\mathbf{S}$ is diagonal and the elements are sorted in descending order,
it is clear that $\mathbf{y} = (0, 0, \dots, 1)^T$.
- ⑤ From $\mathbf{h} = \mathbf{V}\mathbf{y}$, $\mathbf{h}$ is the last column of $\mathbf{V}$.

□ Normalized Coordinates

- Translation of all points to their average point

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} - \begin{pmatrix} \bar{x} \\ \bar{y} \end{pmatrix}$$

- Make their average distance to the origin be $\sqrt{2}$

$$\frac{\sum_{i=1}^N \sqrt{x'^2 + y'^2}}{N} \cdot s = \sqrt{2}, \quad s = \frac{N \cdot \sqrt{2}}{\sum_{i=1}^N \sqrt{x'^2 + y'^2}}$$

- Coordinate Normalization Transform

$$\begin{pmatrix} \hat{x} \\ \hat{y} \end{pmatrix} = s \cdot \begin{pmatrix} x' \\ y' \end{pmatrix} = s \cdot \begin{pmatrix} x \\ y \end{pmatrix} - s \cdot \begin{pmatrix} \bar{x} \\ \bar{y} \end{pmatrix} \quad \Rightarrow \quad \begin{pmatrix} \hat{x} \\ \hat{y} \\ 1 \end{pmatrix} = \begin{pmatrix} s & 0 & -s\bar{x} \\ 0 & s & -s\bar{y} \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

□ Homography with Normalization

- Homography for the normalized coordinates

$$\mathbf{U}_n = \mathbf{H}_n \cdot \mathbf{M}_n$$

Considering the normalizing transforms,

$$\mathbf{T}_U \mathbf{U} = \mathbf{H}_n \cdot \mathbf{T}_M \mathbf{M}$$

$$\mathbf{U} = \mathbf{T}_U^{-1} \mathbf{H}_n \cdot \mathbf{T}_M \mathbf{M}$$

$$\therefore \mathbf{H} = \mathbf{T}_U^{-1} \mathbf{H}_n \mathbf{T}_M$$

□ Two equation of 6 parameters

- $\mathbf{H}$ is defined up to a scale factor

$$\mathbf{H} = (\mathbf{h}_1 \quad \mathbf{h}_2 \quad \mathbf{h}_3)$$

$$(\mathbf{h}_1 \quad \mathbf{h}_2 \quad \mathbf{h}_3) = \lambda \mathbf{K}(\mathbf{r}_1 \quad \mathbf{r}_2 \quad \mathbf{t})$$

defined up to scale

- Orthonormality gives two equations from one image

$$\mathbf{r}_1 \cdot \mathbf{r}_2 = \mathbf{r}_1^T \mathbf{r}_2 = 0$$

$$\|\mathbf{r}_1\|^2 = \|\mathbf{r}_2\|^2$$



$$\mathbf{h}_1^T \mathbf{K}^{-T} \mathbf{K}^{-1} \mathbf{h}_2 = 0$$

$$\mathbf{h}_1^T \mathbf{K}^{-T} \mathbf{K}^{-1} \mathbf{h}_1 = \mathbf{h}_2^T \mathbf{K}^{-T} \mathbf{K}^{-1} \mathbf{h}_2$$

□ Linear Equations

symmetric

$$\text{Letting } \mathbf{B} = \mathbf{K}^{-T} \mathbf{K}^{-1} = \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{21} & B_{22} & B_{23} \\ B_{31} & B_{32} & B_{33} \end{bmatrix}$$

and $\mathbf{b} = [B_{11} \quad B_{12} \quad B_{22} \quad B_{13} \quad B_{23} \quad B_{33}]$ gives,

$$\mathbf{h}_i = [h_{1i} \quad h_{2i} \quad h_{3i}]^T$$

$$\mathbf{h}_i^T \mathbf{B} \mathbf{h}_j = \mathbf{v}_{ij}^T \mathbf{b}$$

$$\mathbf{v}_{ij} = [h_{1i}h_{1j} \quad h_{1i}h_{2j} + h_{2i}h_{1j} \quad h_{2i}h_{2j} \quad h_{3i}h_{1j} + h_{1i}h_{3j} \quad h_{3i}h_{2j} + h_{2i}h_{3j} \quad h_{3i}h_{3j}]^T$$

□ Linear Equations

- Two fundamental constraints become

$$\begin{bmatrix} \mathbf{v}_{12}^T \\ (\mathbf{v}_{11} - \mathbf{v}_{22})^T \end{bmatrix} \mathbf{b} = 0 \quad \Rightarrow \quad \mathbf{V}\mathbf{b} = 0 \quad \text{where } \mathbf{V} \text{ is } 2N \times 6 \text{ matrix}$$

$\mathbf{h}_1^T \mathbf{K}^{-T} \mathbf{K}^{-1} \mathbf{h}_2 = 0$

$\mathbf{h}_1^T \mathbf{K}^{-T} \mathbf{K}^{-1} \mathbf{h}_1 = \mathbf{h}_2^T \mathbf{K}^{-T} \mathbf{K}^{-1} \mathbf{h}_2$

- Solution of $\mathbf{V}\mathbf{b} = 0$ is the eigenvector of $\mathbf{V}^T \mathbf{V}$ associated with the smallest eigenvalue

□ Closed-Form Solution

- If $N \geq 3$, unique solution $\mathbf{b}$ defined up to a scale factor
- If $N = 2$, impose skewless constraint $\gamma = 0$
- If $N = 1$, can only solve two camera intrinsic parameters, α and β , assuming u_0 and v_0 are known and $\gamma = 0$

$$\mathbf{K} = \begin{bmatrix} \alpha & \gamma & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

□ Intrinsic Parameters

- $\mathbf{B}$ is constructed by $\mathbf{b}$
- From $\mathbf{B} = \lambda \mathbf{K}^{-T} \mathbf{K}^{-1}$, intrinsic parameters are then

$$v_0 = \frac{(B_{12}B_{13} - B_{11}B_{23})}{(B_{11}B_{22} - B_{12}^2)}$$

$$\lambda = B_{33} - \frac{[B_{13}^2 + v_0(B_{12}B_{13} - B_{11}B_{23})]}{B_{11}}$$

$$\alpha = \sqrt{\frac{\lambda}{B_{11}}}, \quad \beta = \sqrt{\frac{\lambda B_{11}}{(B_{11}B_{22} - B_{12}^2)}}$$

$$\gamma = -\frac{B_{12}\alpha^2\beta}{\lambda}, \quad u_0 = \frac{\gamma v_0}{\alpha} - \frac{B_{13}\alpha^2}{\lambda}$$

□ Extrinsic Parameters

- Calculating extrinsic parameter from $\mathbf{H} = [\mathbf{h}_1 \ \mathbf{h}_2 \ \mathbf{h}_3] = \lambda \mathbf{K}[\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}]$,

$$\mathbf{r}_1 = \frac{\mathbf{K}^{-1}\mathbf{h}_1}{\lambda} \quad \mathbf{r}_2 = \frac{\mathbf{K}^{-1}\mathbf{h}_2}{\lambda} \quad \mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2 \quad \mathbf{t} = \frac{\mathbf{K}^{-1}\mathbf{h}_3}{\lambda}$$

$$\lambda = \|\mathbf{K}^{-1}\mathbf{h}_1\| \text{ or } \|\mathbf{K}^{-1}\mathbf{h}_2\|$$

- $\mathbf{R} = [\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{r}_3]$ does not, in general, satisfy properties of a rotation matrix because of noise in data $\rightarrow$ should be orthogonalized !
- $\mathbf{R}$ can be orthogonalized through singular value decomposition

$$\mathbf{R} = \mathbf{U}\mathbf{S}\mathbf{V}^T$$

$$\hat{\mathbf{R}} = \mathbf{U}\mathbf{V}^T$$

orthogonalized rotation

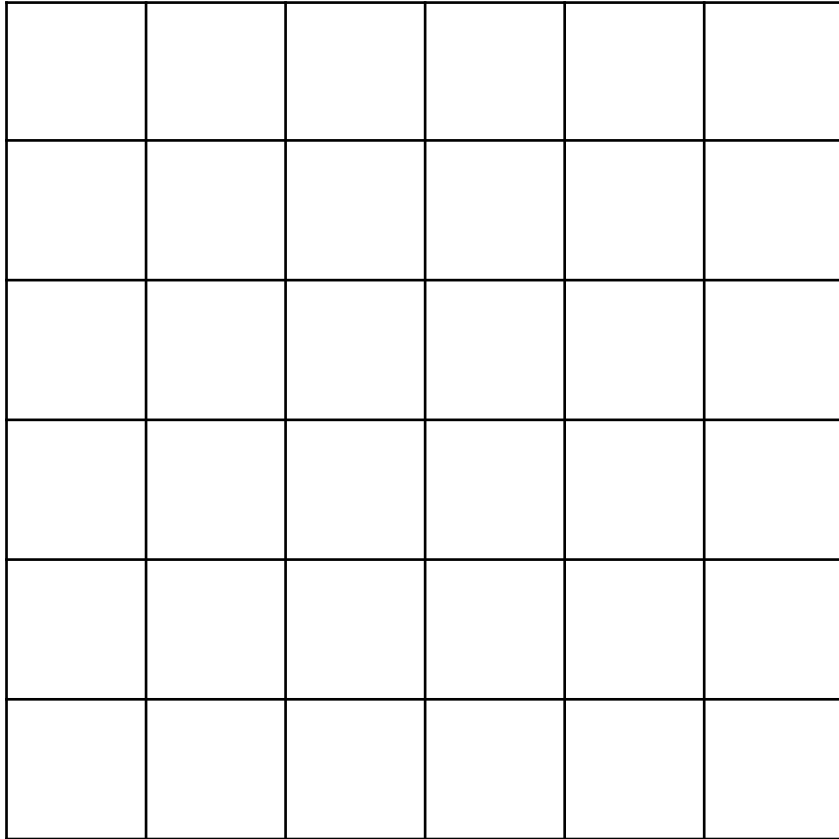
Camera Calibration

- Camera Geometry
- Zhang's Method
- Fine-Tuning of Intrinsic Parameters

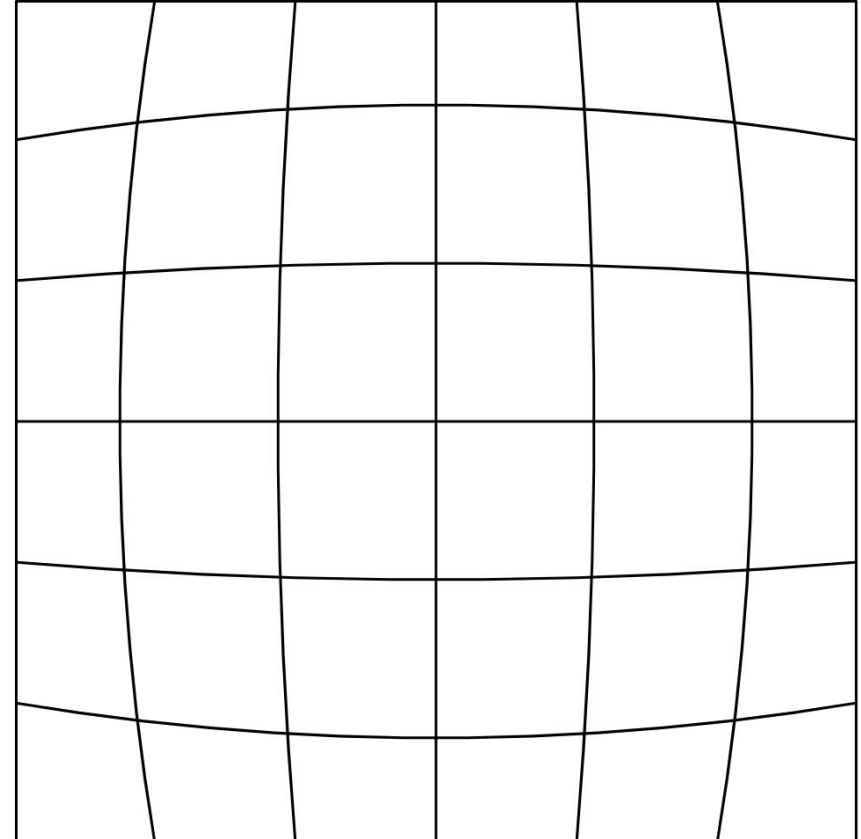


❑ Lens Distortion

- Brown Model(1966)
- Fitzgibbon Model(2001)



ideal



distorted

❑ Lens Distortion

- Brown Model(1966)
- Fitzgibbon Model(2001)



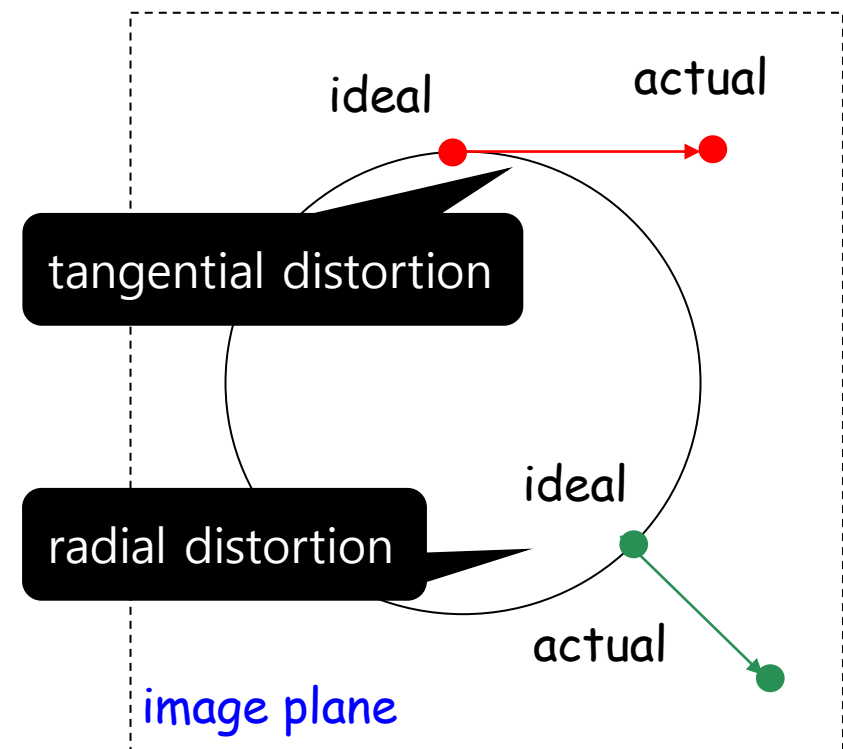
□ Fitzgibbon Model

- Fitzgibbon, A. W., "Simultaneous linear estimation of multiple view geometry and lens distortion", 2001 CVPR
- More accurate

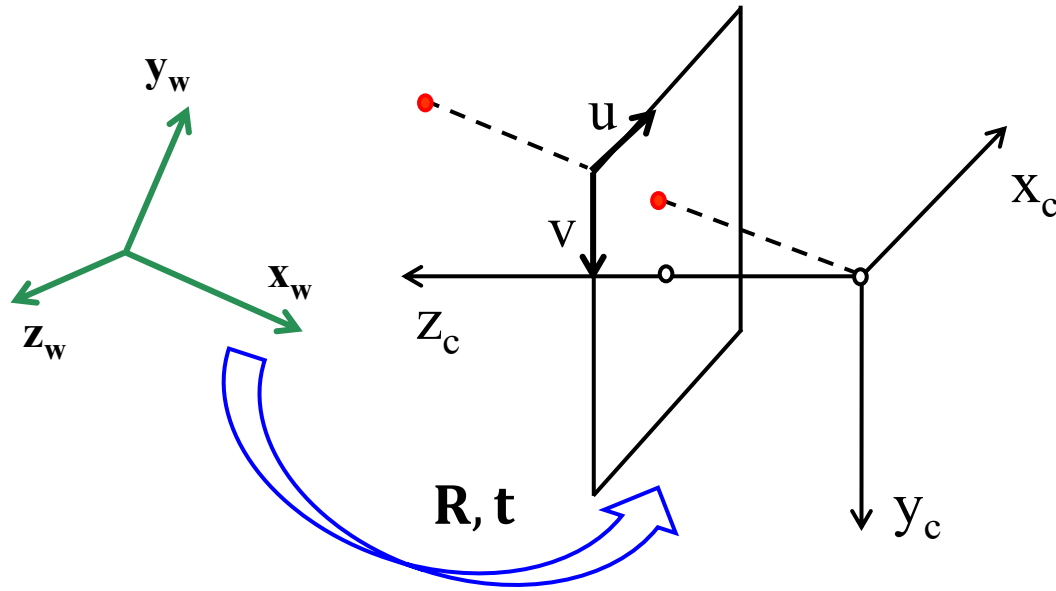
$$s \begin{pmatrix} r_x \\ r_y \\ 1 \end{pmatrix} = R \begin{pmatrix} x_w \\ y_w \\ z_w \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \\ t_z \end{pmatrix}, \quad r = \sqrt{r_x^2 + r_y^2}$$

$$\textcircled{1} \quad \begin{pmatrix} u_d \\ v_d \\ 1 \end{pmatrix} = \mathbf{K} \begin{pmatrix} r_x(1 + k_1 r^2 + k_2 r^4) \\ r_y(1 + k_1 r^2 + k_2 r^4) \\ 1 \end{pmatrix}$$

$$\textcircled{2} \quad \begin{cases} u_d - u_0 = (u - u_0)(1 + k_1 r^2 + k_2 r^4) \\ v_d - v_0 = (v - v_0)(1 + k_1 r^2 + k_2 r^4) \end{cases}$$



□ From Scene To Image



$$\tilde{\mathbf{X}}_c = \begin{pmatrix} x_c \\ y_c \\ z_c \\ 1 \end{pmatrix} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0 & 1 \end{bmatrix} \begin{pmatrix} x_w \\ y_w \\ z_w \\ 1 \end{pmatrix} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0 & 1 \end{bmatrix} \tilde{\mathbf{X}}_w$$

$$\begin{pmatrix} U \\ V \\ S \end{pmatrix} = \begin{bmatrix} f & 0 & u_o \\ 0 & f & v_o \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{pmatrix} x_c \\ y_c \\ z_c \\ 1 \end{pmatrix}$$

$$\tilde{\mathbf{U}} = \mathbf{K} [\mathbf{I} | 0] \tilde{\mathbf{X}}_c = \mathbf{K} [\mathbf{R} | \mathbf{t}] \tilde{\mathbf{X}}_w$$

$$\tilde{\mathbf{U}} = \mathbf{P} \tilde{\mathbf{X}}_w, \mathbf{P} = \mathbf{K} [\mathbf{R} | \mathbf{t}]$$

$$u = U/S$$

$$v = V/S$$

$$u_d - u_0 = (u - u_0)(1 + k_1 r^2 + k_2 r^4)$$

$$v_d - v_0 = (v - v_0)(1 + k_1 r^2 + k_2 r^4)$$

□ From Scene To Image

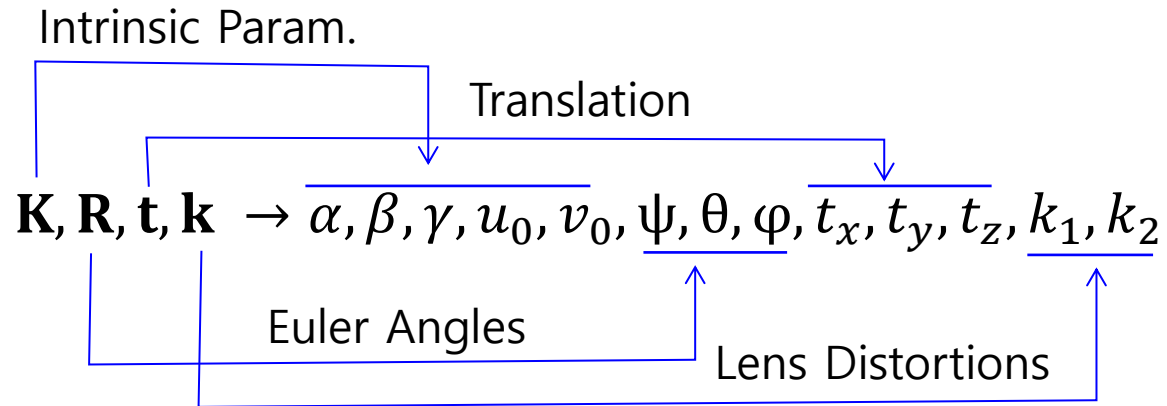
$$(u_d, v_d) = \text{projection}(x_w, y_w, z_w)$$

$$\begin{pmatrix} x_w \\ y_w \\ z_w = 0 \end{pmatrix} \xrightarrow{\mathbf{R}, \mathbf{t}} \begin{pmatrix} x_c \\ y_c \\ z_c \end{pmatrix} \xrightarrow{\mathbf{K}} \begin{pmatrix} u \\ v \end{pmatrix} \xrightarrow{k_1, k_2} \begin{pmatrix} u_d \\ v_d \end{pmatrix}$$

$$\begin{aligned} \tilde{\mathbf{U}} &= \mathbf{K} [\mathbf{R} \ \mathbf{t}] \tilde{\mathbf{X}}_w \\ &= \mathbf{H} \tilde{\mathbf{X}}_w \end{aligned}$$

□ Fine-Tuning

- Target parameters including lens distortions



For N captures,

$$(\mathbf{K}, \mathbf{k}), (\mathbf{R}, \mathbf{t})_i \rightarrow (\alpha, \beta, \gamma, u_0, v_0, k_1, k_2), (\psi, \theta, \varphi, t_x, t_y, t_z)_i, \quad i = 1, \dots, N$$

❑ Nonlinear Optimization

- Cost function

number of frames

number of target points

projection of image i

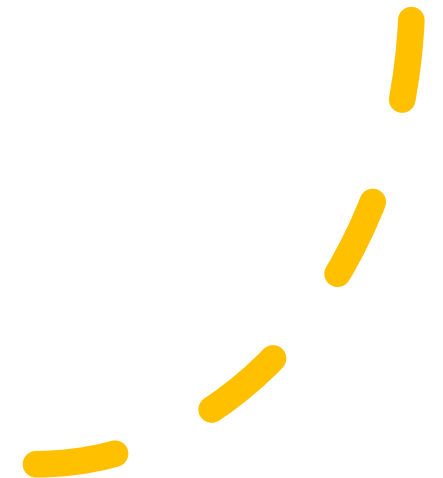
$$Error = \sum_{i=1}^N \sum_{k=1}^{K_i} \| \mathbf{u}_{ik} - \mathbf{u}_d^{ik} \|^2, \quad \mathbf{u}_d^{ik} = p_i(x_w^{(k)}, y_w^{(k)})$$

coordinates of feature k in image i

- Python module: scipy
 - ✓ from scipy.optimize import minimize → local optimization
 - ✓ from scipy.optimize import basinhopping → global optimization
- Ceres Solver

Robot-Eye Calibration

- Robot-Eye Calibration
- Tsai & Lenz Method
- Dornaika & Horaud Method
- Non-linear Optimization

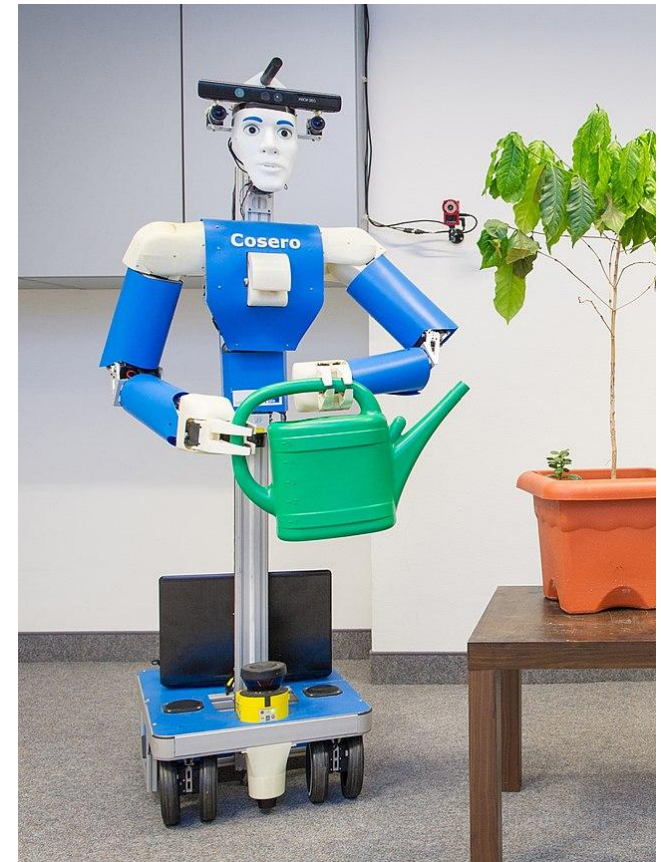


□ Visual Manipulation

- A fundamental skill that enables a robot to perform a variety of tasks
 - ✓ Industrial automation
 - ✓ Assistance in medical operations
 - ✓ Robotic grasping (i.e., service robots).



<https://www.idtechex.com>



https://www.wikiwand.com/en/Service_robot

□ Three Calibrations for Visual Manipulation

- Camera Calibration

K : Camera matrix

- Robot-Eye Calibration

- 1) Eye-in-Hand Calibration

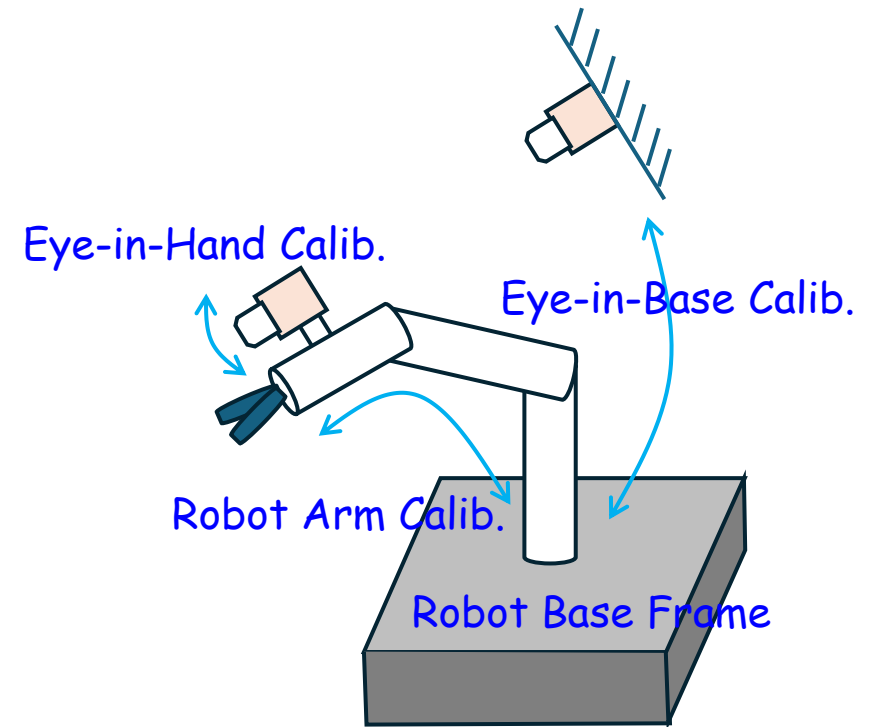
T_h^c : Homogeneous transformation
from the hand frame to the camera frame

- 2) Eye-to-Hand Calibration

T_b^c : Homogeneous transformation
from the robot base frame to the camera frame

- Robot Arm Calibration

$T_{J\#}^b$: Homogeneous transformation between the robot base frame
to the joint frames



□ Eye-in-Hand Calibration

- $AX = XB$

1) $X = X_c^h$

$$A_{h_1}^b X_{c_1}^{h_1} B_w^{c_1} = A_{h_2}^b X_{c_2}^{h_2} B_w^{c_2}$$

$$A_b^{h_2} A_{h_1}^b X_{c_1}^{h_1} = X_{c_2}^{h_2} B_w^{c_2} B_{c_1}^w$$

$$A_{h_1}^{h_2} X = XB_{c_1}^{c_2}$$

$$AX = XB$$

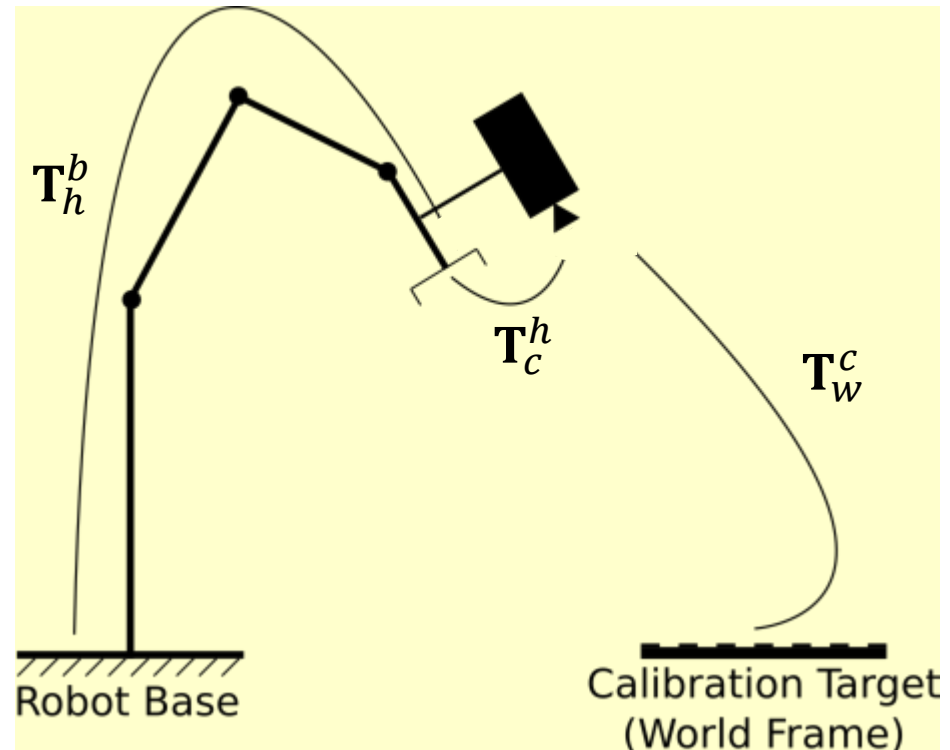
2) $X = X_h^c$

$$A_{c_1}^w X_{h_1}^{c_1} B_b^{h_1} = A_{c_2}^w X_{h_2}^{c_2} B_b^{h_2}$$

$$A_w^{c_2} A_{c_1}^w X_{h_1}^{c_1} = X_{h_2}^{c_2} B_b^{h_2} B_{h_1}^b$$

$$A_{c_1}^{c_2} X = XB_{h_1}^{h_2}$$

$$AX = XB$$



→ To estimate X using corresponding interframe poses of the robot hand and the camera

□ Eye-in-Hand Calibration

- $AX = ZB$

From the figure,

$$\mathbf{T}_{c_i}^w \mathbf{T}_{h_i}^{c_i} = \mathbf{T}_b^w \mathbf{T}_{h_i}^b.$$

Letting the knowns

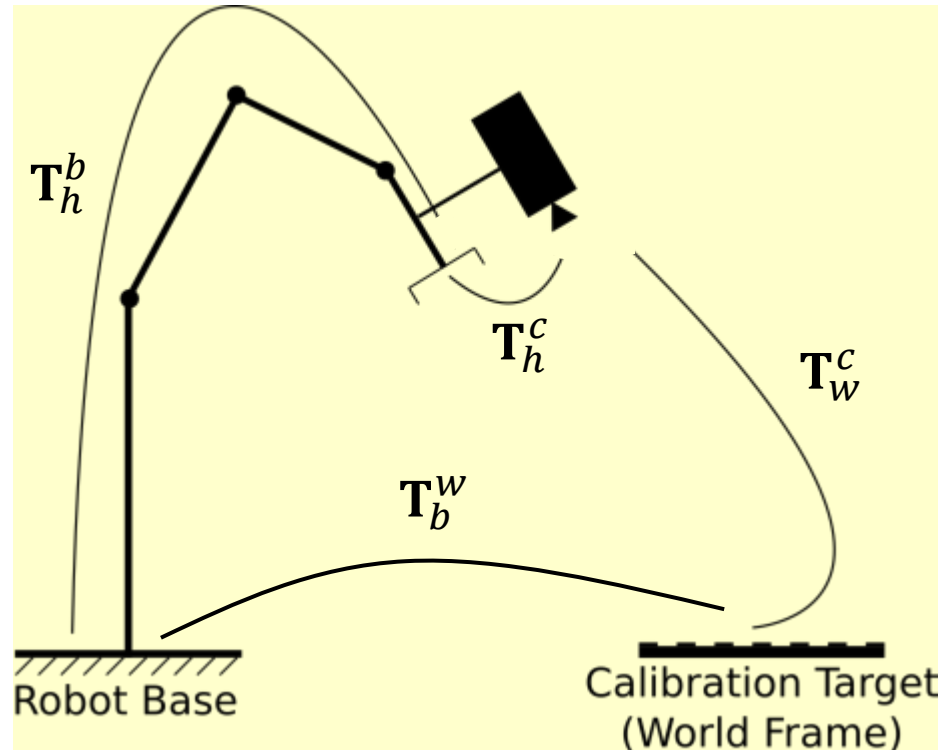
$$\mathbf{T}_{c_i}^w = \mathbf{A}_i \text{ and } \mathbf{T}_{h_i}^b = \mathbf{B}_i,$$

and the unknowns

$$\mathbf{T}_{h_i}^{c_i} = \mathbf{T}_h^c = \mathbf{X} \text{ and } \mathbf{T}_b^w = \mathbf{Z},$$

it follows

$$\mathbf{A}_i \mathbf{X} = \mathbf{Z} \mathbf{B}_i, \quad i = 1, \dots, N.$$



→ To estimate $\mathbf{X}$ and $\mathbf{Z}$ using corresponding poses of the robot hand and the camera

□ Eye-in-Hand Calibration

- $AX = ZB$

From the figure,

$$\mathbf{T}_{c_i}^w \mathbf{T}_{h_i}^{c_i} = \mathbf{T}_b^w \mathbf{T}_{h_i}^b \quad \longrightarrow \quad \mathbf{T}_w^{c_i} \mathbf{T}_b^w = \mathbf{T}_{h_i}^{c_i} \mathbf{T}_b^{h_i}.$$

Letting the knowns

$$\mathbf{T}_{c_i}^w = \mathbf{A}_i \text{ and } \mathbf{T}_{h_i}^b = \mathbf{B}_i, \quad \longrightarrow \quad \mathbf{T}_w^{c_i} = \mathbf{A} \text{ and } \mathbf{T}_b^{h_i} = \mathbf{B}$$

and the unknowns

$$\mathbf{T}_{h_i}^{c_i} = \mathbf{T}_h^c = \mathbf{X} \text{ and } \mathbf{T}_b^w = \mathbf{Z}, \quad \longrightarrow \quad \mathbf{T}_b^w = \mathbf{X} \text{ and } \mathbf{T}_{h_i}^{c_i} = \mathbf{T}_h^c = \mathbf{Z}$$

it follows

$$\mathbf{A}_i \mathbf{X} = \mathbf{Z} \mathbf{B}_i, \quad i = 1, \dots, N.$$

→ To estimate $\mathbf{X}$ and $\mathbf{Z}$ using corresponding poses of the robot hand and the camera

□ Eye-in-Hand Calibration

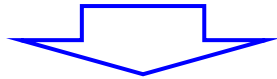
- Classical approach

$$\mathbf{AX} = \mathbf{XB}$$

$$\begin{pmatrix} \mathbf{R}_A & \mathbf{t}_A \\ \mathbf{0} & 1 \end{pmatrix} \begin{pmatrix} \mathbf{R}_X & \mathbf{t}_X \\ \mathbf{0} & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{R}_X & \mathbf{t}_X \\ \mathbf{0} & 1 \end{pmatrix} \begin{pmatrix} \mathbf{R}_B & \mathbf{t}_B \\ \mathbf{0} & 1 \end{pmatrix}$$

$$\begin{pmatrix} \mathbf{R}_A \mathbf{R}_X & \mathbf{R}_A \mathbf{t}_X + \mathbf{t}_A \\ \mathbf{0} & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{R}_X \mathbf{R}_B & \mathbf{R}_X \mathbf{t}_B + \mathbf{t}_X \\ \mathbf{0} & 1 \end{pmatrix}$$

$$\therefore \begin{cases} \mathbf{R}_A \mathbf{R}_X = \mathbf{R}_X \mathbf{R}_B \\ \mathbf{R}_A \mathbf{t}_X + \mathbf{t}_A = \mathbf{R}_X \mathbf{t}_B + \mathbf{t}_X \end{cases}$$



Separated solution : $\mathbf{R}_X \rightarrow \mathbf{t}_X$
Simultaneous solution : $\mathbf{R}_X, \mathbf{t}_X$



Non-linear optimization using
Reprojection error minimization

□ Eye-to-Hand Calibration

- $AX = XB$

From the figure,

$$\mathbf{T}_b^{h_i} \mathbf{T}_c^b \mathbf{T}_{o_i}^c = \mathbf{T}_b^{h_j} \mathbf{T}_c^b \mathbf{T}_{o_j}^c$$

$$\mathbf{T}_{h_j}^b \mathbf{T}_b^{h_i} \mathbf{T}_c^b = \mathbf{T}_c^b \mathbf{T}_{o_j}^c \mathbf{T}_c^{o_i}$$

Letting the knowns

$$\mathbf{T}_{h_j}^b \mathbf{T}_b^{h_i} = \mathbf{A} \text{ and } \mathbf{T}_{o_j}^c \mathbf{T}_c^{o_i} = \mathbf{B},$$

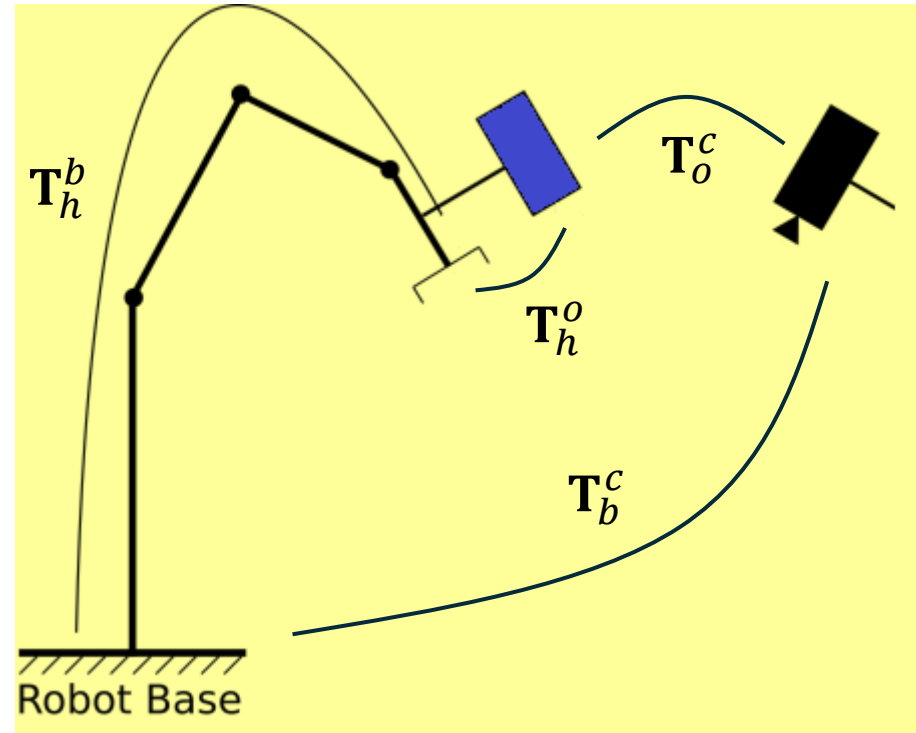
and the unknowns

$$\mathbf{T}_c^b = \mathbf{X},$$

it follows

$$\mathbf{A}_i \mathbf{X} = \mathbf{X} \mathbf{B}_i, \quad i = 1, \dots, N.$$

※ $\mathbf{T}_{o_i}^c$: 2D pattern, 3D pattern or Gripper (3D camera)



□ Eye-to-Hand Calibration

- $AX = ZB$

From the figure,

$$\mathbf{T}_{o_i}^c \mathbf{T}_{h_i}^{o_i} = \mathbf{T}_b^c \mathbf{T}_{h_i}^b$$

Letting the knowns

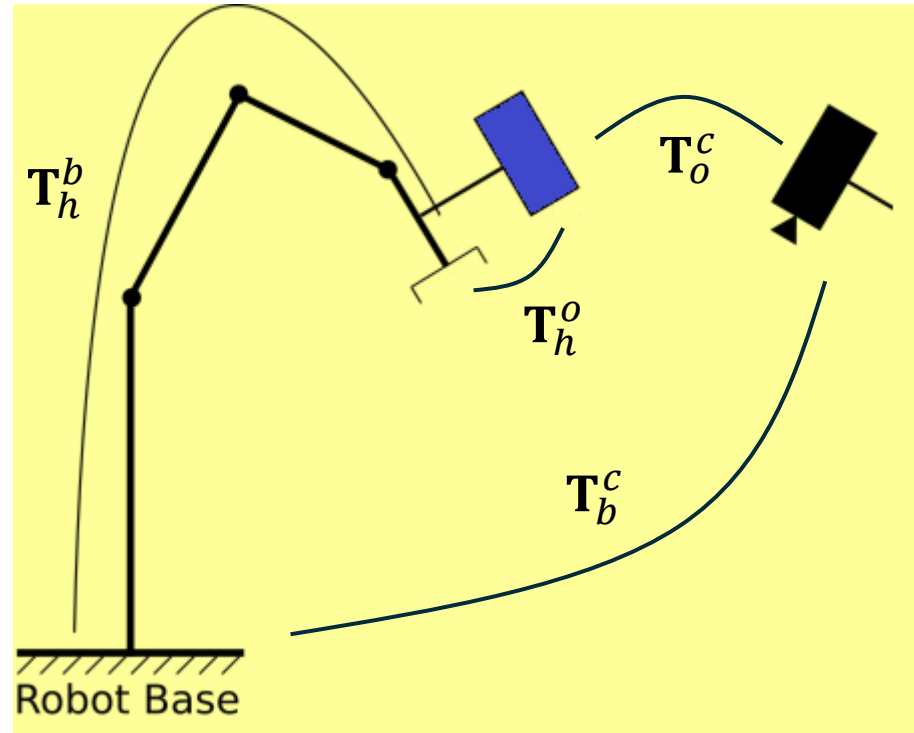
$$\mathbf{T}_{o_i}^c = \mathbf{A}_i \text{ and } \mathbf{T}_{h_i}^b = \mathbf{B}_i,$$

and the unknowns

$$\mathbf{T}_{h_i}^{o_i} = \mathbf{T}_h^o = \mathbf{X} \text{ and } \mathbf{T}_b^c = \mathbf{Z},$$

it follows

$$\mathbf{A}_i \mathbf{X} = \mathbf{Z} \mathbf{B}_i, \quad i = 1, \dots, N.$$



※ $\mathbf{T}_{o_i}^c$: 2D pattern, 3D pattern or Gripper (3D camera)

□ Eye-to-Hand Calibration

- $\mathbf{AX} = \mathbf{ZB}$

From the figure,

$$\mathbf{T}_{o_i}^c \mathbf{T}_{h_i}^{o_i} = \mathbf{T}_b^c \mathbf{T}_{h_i}^b \quad \longrightarrow \quad \mathbf{T}_c^{o_i} \mathbf{T}_b^c = \mathbf{T}_{h_i}^{o_i} \mathbf{T}_b^{h_i}$$

Letting the knowns

$$\mathbf{T}_{o_i}^c = \mathbf{A}_i \text{ and } \mathbf{T}_{h_i}^b = \mathbf{B}_i, \quad \longrightarrow \quad \mathbf{T}_c^{o_i} = \mathbf{A} \text{ and } \mathbf{T}_b^{h_i} = \mathbf{B}$$

and the unknowns

$$\mathbf{T}_{h_i}^{o_i} = \mathbf{T}_h^o = \mathbf{X} \text{ and } \mathbf{T}_b^c = \mathbf{Z}, \quad \longrightarrow \quad \mathbf{T}_b^c = \mathbf{X} \text{ and } \mathbf{T}_{h_i}^{o_i} = \mathbf{T}_h^o = \mathbf{Z}$$

it follows

$$\mathbf{A}_i \mathbf{X} = \mathbf{ZB}_i, \quad i = 1, \dots, N.$$

※ $\mathbf{T}_{o_i}^c$: 2D pattern, 3D pattern or Gripper (3D camera)

Robot-Eye Calibration

- Robot-Eye Calibration
- Tsai & Lenz Method
- Dornaika & Horaud Method
- Non-linear Optimization



□ Tsai & Lenz Method

- “A New Technique for Fully Autonomous and Efficient 3D Robotics Hand-Eye Calibration”, 1989, IEEE Trans. R. & A.
 - ✓ Classical algorithm ($AX = XB$)
 - ✓ Analytic solution
 - ✓ Rotation by Angle-Axis
 - ✓ Separated solution

$$AX = XB$$

$$\begin{pmatrix} \mathbf{R}_A & \mathbf{t}_A \\ \mathbf{0} & 1 \end{pmatrix} \begin{pmatrix} \mathbf{R}_X & \mathbf{t}_X \\ \mathbf{0} & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{R}_X & \mathbf{t}_X \\ \mathbf{0} & 1 \end{pmatrix} \begin{pmatrix} \mathbf{R}_B & \mathbf{t}_B \\ \mathbf{0} & 1 \end{pmatrix}$$

$$\begin{pmatrix} \mathbf{R}_A \mathbf{R}_X & \mathbf{R}_A \mathbf{t}_X + \mathbf{t}_A \\ \mathbf{0} & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{R}_X \mathbf{R}_B & \mathbf{R}_X \mathbf{t}_B + \mathbf{t}_X \\ \mathbf{0} & 1 \end{pmatrix}$$

$$\therefore \begin{cases} \mathbf{R}_A \mathbf{R}_X = \mathbf{R}_X \mathbf{R}_B \\ \mathbf{R}_A \mathbf{t}_X + \mathbf{t}_A = \mathbf{R}_X \mathbf{t}_B + \mathbf{t}_X \end{cases}$$

Separated solution : $\mathbf{R}_X \rightarrow \mathbf{t}_X$
Simultaneous solution : $\mathbf{R}_X, \mathbf{t}_X$



Non-linear optimization

□ Basic Setup

- Homogeneous Transformation ($\tilde{\mathbf{x}}_c = \mathbf{T}_w^c \tilde{\mathbf{x}}_w$)

✓ $\mathbf{T}_h^b, \mathbf{T}_c^h, \mathbf{T}_w^c$

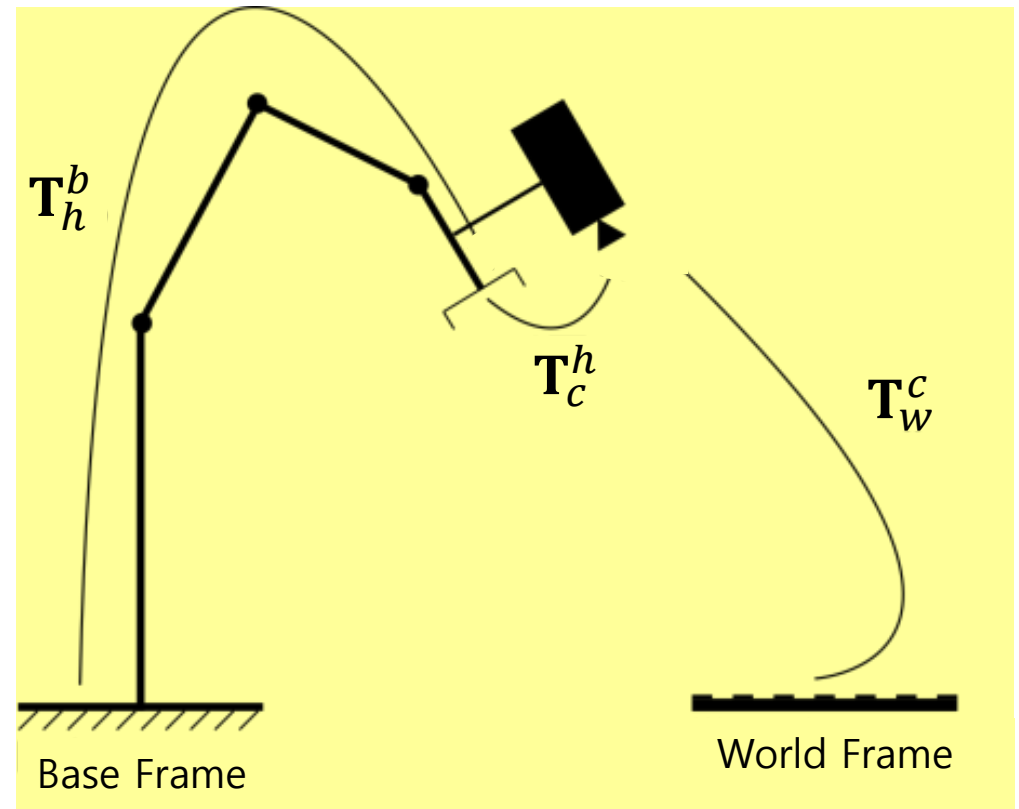
✓ $\mathbf{T}_{c_{ij}} = \mathbf{T}_{c_i}^{c_j} = \mathbf{T}_w^{c_j} \mathbf{T}_w^{c_i^{-1}}$

✓ $\mathbf{T}_{h_{ij}} = \mathbf{T}_{h_i}^{h_j} = \mathbf{T}_{h_j}^b{}^{-1} \mathbf{T}_{h_i}^b$

✓ $\mathbf{T}_{ch} = \mathbf{T}_{c_i}^{h_i}$

✓ $\mathbf{T}_{h_{ij}} \mathbf{X} = \mathbf{X} \mathbf{T}_{c_{ij}}$

$\mathbf{A}_{h_1}^{h_2} \mathbf{X} = \mathbf{X} \mathbf{B}_{c_1}^{c_2}$
 $\mathbf{A} \mathbf{X} = \mathbf{X} \mathbf{B}$



□ Basic Background for Rotation

- Angle-Axis representation of rotation

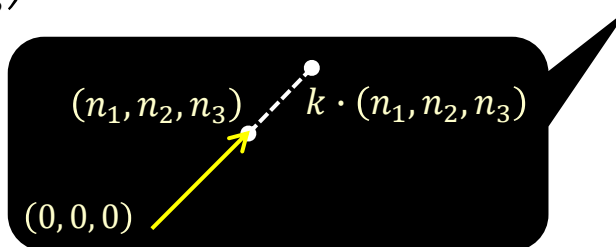
https://en.Wikipedia.org/wiki/Rotation_matrix

- Rotation matrix by the angle θ around the axis (n_1, n_2, n_3) , $\|n_1, n_2, n_3\| = 1$

$$\mathbf{R} = \begin{pmatrix} n_1^2(1 - \cos\theta) + \cos\theta & n_1n_2(1 - \cos\theta) - n_3\sin\theta & n_1n_3(1 - \cos\theta) + n_2\sin\theta \\ n_1n_2(1 - \cos\theta) + n_3\sin\theta & n_2^2(1 - \cos\theta) + \cos\theta & n_2n_3(1 - \cos\theta) - n_1\sin\theta \\ n_1n_3(1 - \cos\theta) - n_2\sin\theta & n_2n_3(1 - \cos\theta) + n_1\sin\theta & n_3^2(1 - \cos\theta) + \cos\theta \end{pmatrix}$$

- $\mathbf{R}$ is a rotation around the axis (n_1, n_2, n_3) , therefore,

$$\mathbf{R} \begin{pmatrix} n_1 \\ n_2 \\ n_3 \end{pmatrix} = \begin{pmatrix} n_1 \\ n_2 \\ n_3 \end{pmatrix} \rightarrow \text{the axis } (n_1, n_2, n_3) \text{ is the eigenvector of the eigenvalue } 1(e^{\pm j\theta})$$



□ Basic Background for Rotation

- Angle-Axis representation of rotation

```
def rot_from_axis_angle(axis, theta):  
    axis = axis/np.linalg.norm(axis)  
    c = np.cos(theta)  
    s = np.sin(theta)  
    t = 1 - c  
    x = axis[0]  
    y = axis[1]  
    z = axis[2]  
    rot = np.zeros((3, 3))  
    rot[0, 0] = t * x * x + c  
    rot[0, 1] = t * x * y - s * z  
    rot[0, 2] = t * z * x + s * y  
    rot[1, 0] = t * x * y + s * z  
    rot[1, 1] = t * y * y + c  
    rot[1, 2] = t * y * z - s * x  
    rot[2, 0] = t * z * x - s * y  
    rot[2, 1] = t * y * z + s * x  
    rot[2, 2] = t * z * z + c  
    return rot
```

□ Basic Background for Rotation

- Using Rodrigues formula, the $\mathbf{R}$ can be represented by $\mathbf{p}_r$

$$\mathbf{p}_r = 2\sin\frac{\theta}{2}(n_1, n_2, n_3)^T, \quad 0 \leq \theta \leq \pi$$

scaled rotation axis

$$\mathbf{R} = \left(1 - \frac{|\mathbf{p}_r|^2}{2}\right) \mathbf{I} + \frac{1}{2}(\mathbf{p}_r \mathbf{p}_r^T + \alpha \cdot \text{skew}(\mathbf{p}_r)),$$

$$\text{where } \alpha = \sqrt{4 - |\mathbf{p}_r|^2}, \quad \text{skew}((a, b, c)^T) = \begin{pmatrix} 0 & -c & b \\ c & 0 & -a \\ -b & a & 0 \end{pmatrix}$$

□ Basic Background for Rotation

- Principal axis = scaled rotation axis

$\mathbf{p}_{h_{ij}}$ principal axis for $\mathbf{R}_{h_{ij}}$ ($\leftarrow \mathbf{T}_{h_{ij}}$)

$\mathbf{p}_{c_{ij}}$ principal axis for $\mathbf{R}_{c_{ij}}$ ($\leftarrow \mathbf{T}_{c_{ij}}$)

$\mathbf{p}_{ch}$ principal axis for $\mathbf{R}_{ch}$ ($\leftarrow \mathbf{T}_{ch}$)

$$\mathbf{p}'_{ch} = \frac{1}{\sqrt{4 - |\mathbf{p}_{ch}|^2}} \mathbf{p}_{ch}$$

□ Basic Background for Rotation

- Similarity Transformation

In the original coordinate system, $\mathbf{y} = \mathbf{T}_{n \times n} \mathbf{x}$.

In the new coordinate system, $\mathbf{y}' = \mathbf{S}_{n \times n} \mathbf{x}'$.

Considering the transformation $\mathbf{P}$ like

$$\mathbf{x}' = \mathbf{P} \mathbf{x}, \mathbf{y}' = \mathbf{P} \mathbf{y},$$

gives

$$\mathbf{P} \mathbf{y} = \mathbf{S} \mathbf{P} \mathbf{x}.$$

Then,

$$\mathbf{y} = \mathbf{P}^{-1} \mathbf{S} \mathbf{P} \mathbf{x}.$$

$$\therefore \mathbf{T} = \mathbf{P}^{-1} \mathbf{S} \mathbf{P}$$

$\mathbf{T}, \mathbf{S}$ are similar



$$\mathbf{S} \xrightarrow{\text{Similarity Tr.}} \mathbf{P} \mathbf{T} \mathbf{P}^{-1}$$

□ Tsai & Lenz Method

- To obtain $\mathbf{R}_{ch}$ and $\mathbf{t}_{ch}$, separately

$$\mathbf{T}_{hij}\mathbf{X} = \mathbf{X}\mathbf{T}_{cij}, \quad \mathbf{X} = \mathbf{T}_{ch}$$

$$\begin{cases} \mathbf{R}_A\mathbf{R}_X = \mathbf{R}_X\mathbf{R}_B \\ \mathbf{R}_A\mathbf{t}_X + \mathbf{t}_A = \mathbf{R}_X\mathbf{t}_B + \mathbf{t}_X \end{cases} \Rightarrow \begin{cases} \mathbf{R}_{hij}\mathbf{R}_{ch} = \mathbf{R}_{ch}\mathbf{R}_{cij} \\ \mathbf{R}_{hij}\mathbf{t}_{ch} + \mathbf{t}_{hij} = \mathbf{R}_{ch}\mathbf{t}_{cij} + \mathbf{t}_{ch} \end{cases}$$

- ① Compute $\mathbf{p}'_{ch}$ by least squares technique from

$$\text{skew}(\mathbf{p}_{hij} + \mathbf{p}_{cij})\mathbf{p}'_{ch} = \mathbf{p}_{hij} - \mathbf{p}_{cij} \quad \text{for every pair of frames } i, j$$

- The interframe angle (rotation angle $\mathbf{R}_{cij}$ or $\mathbf{R}_{hij}$) should be as large as possible
- At least two pairs of frames

$$\mathbf{P}_k\mathbf{p}'_{ch} = \mathbf{Q}_k \rightarrow \begin{pmatrix} \mathbf{P}_1 \\ \vdots \\ \mathbf{P}_K \end{pmatrix} \mathbf{p}'_{ch} = \begin{pmatrix} \mathbf{Q}_1 \\ \vdots \\ \mathbf{Q}_K \end{pmatrix}, \quad \mathbf{p}'_{ch} = \mathbf{P}^+\mathbf{Q} = \mathbf{V}\mathbf{W}^+\mathbf{U}^T\mathbf{Q} \quad \text{where } \mathbf{P} = \mathbf{U}\mathbf{W}\mathbf{V}^T$$

$$\mathbf{P}_{m \times n} = \mathbf{U} \begin{pmatrix} e_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & e_n \\ \vdots & \vdots & \vdots \\ 0 & 0 & 0 \end{pmatrix} \mathbf{V}^T \quad \Rightarrow \quad \mathbf{P}^+ = \mathbf{V} \begin{pmatrix} 1/e_1 & & & \\ & \ddots & & \\ & & 1 & \dots 0 \\ & & e_n & \end{pmatrix} \mathbf{U}^T$$

□ Tsai & Lenz Method

② Compute $\mathbf{p}_{ch}$

$$\mathbf{p}_{ch} = \frac{2\mathbf{p}'_{ch}}{\sqrt{1+|\mathbf{p}'_{ch}|^2}} \quad \leftarrow \quad \mathbf{p}'_{ch} = \frac{1}{\sqrt{4-|\mathbf{p}_{ch}|^2}} \mathbf{p}_{ch}$$

③ Compute $\mathbf{t}_{ch}$ by least squares technique from

$$(\mathbf{R}_{hij} - \mathbf{I})\mathbf{t}_{ch} = \mathbf{R}_{ch}\mathbf{t}_{cij} - \mathbf{t}_{hij} \quad \text{for every pair of frames } i, j$$

$$\mathbf{P}_k \mathbf{t}_{ch} = \mathbf{Q}_k \quad \rightarrow \quad \begin{pmatrix} \mathbf{P}_1 \\ \vdots \\ \mathbf{P}_K \end{pmatrix} \mathbf{t}_{ch} = \begin{pmatrix} \mathbf{Q}_1 \\ \vdots \\ \mathbf{Q}_K \end{pmatrix}, \quad \mathbf{t}_{ch} = \mathbf{V}\mathbf{W}^+\mathbf{U}^T\mathbf{Q} \quad \text{where } \mathbf{P} = \mathbf{U}\mathbf{W}\mathbf{V}^T$$

$$\begin{cases} \mathbf{R}_{hij}\mathbf{R}_{ch} = \mathbf{R}_{ch}\mathbf{R}_{cij} \\ \mathbf{R}_{hij}\mathbf{t}_{ch} + \mathbf{t}_{hij} = \mathbf{R}_{ch}\mathbf{t}_{cij} + \mathbf{t}_{ch} \end{cases}$$

$$\square \text{ skew}(\mathbf{p}_{h_{ij}} + \mathbf{p}_{c_{ij}}) \mathbf{p}'_{ch} = \mathbf{p}_{h_{ij}} - \mathbf{p}_{c_{ij}}$$

- $\mathbf{T}_{h_i}^{h_j} = \mathbf{T}_{c_j}^{h_j} \mathbf{T}_{c_i}^{c_j} \mathbf{T}_{h_i}^{c_i}$ gives

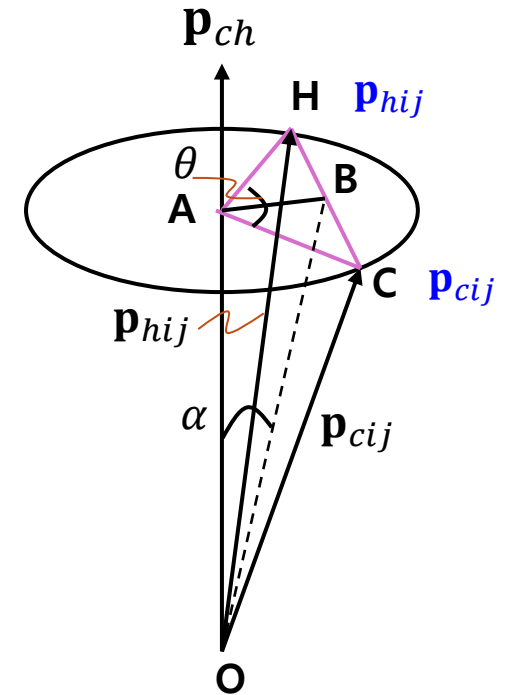
$$\mathbf{R}_{h_{ij}} = \mathbf{R}_{ch} \mathbf{R}_{c_{ij}} \mathbf{R}_{ch}^T.$$

As a result,

$$\mathbf{p}_{h_{ij}} = \mathbf{R}_{ch} \mathbf{p}_{c_{ij}}.$$

- $$\begin{aligned} (\mathbf{p}_{h_{ij}} - \mathbf{p}_{c_{ij}})^T \mathbf{p}_{ch} &= (\mathbf{p}_{h_{ij}} - \mathbf{p}_{c_{ij}})^T \mathbf{R}_{ch}^T \mathbf{R}_{ch} \mathbf{p}_{ch} \\ &= (\mathbf{R}_{ch} \mathbf{R}_{ch}^T \mathbf{p}_{h_{ij}} - \mathbf{R}_{ch} \mathbf{R}_{ch}^T \mathbf{p}_{c_{ij}})^T \mathbf{p}_{ch} \\ &= (\mathbf{R}_{ch} \mathbf{p}_{c_{ij}} - \mathbf{p}_{c_{ij}})^T \mathbf{p}_{ch} \quad (\because \mathbf{p}_{h_{ij}} = \mathbf{R}_{ch} \mathbf{p}_{c_{ij}}) \\ &= \mathbf{p}_{c_{ij}}^T (\mathbf{R}_{ch} - \mathbf{I})^T \mathbf{p}_{ch} \\ &= \mathbf{p}_{c_{ij}}^T (\mathbf{R}_{ch}^T \mathbf{p}_{ch} - \mathbf{p}_{ch}) = 0 \quad (\because \mathbf{R} \mathbf{p} = \mathbf{p}) \end{aligned}$$

$$\therefore \mathbf{p}_{ch} \perp (\mathbf{p}_{h_{ij}} - \mathbf{p}_{c_{ij}})$$



$$\square \text{ skew}(\mathbf{p}_{h_{ij}} + \mathbf{p}_{c_{ij}}) \mathbf{p}'_{ch} = \mathbf{p}_{h_{ij}} - \mathbf{p}_{c_{ij}}$$

- $\mathbf{T}_{h_i}^{h_j} = \mathbf{T}_{c_j}^{h_j} \mathbf{T}_{c_i}^{c_j} \mathbf{T}_{h_i}^{c_i}$ gives

$$\mathbf{R}_{h_{ij}} = \mathbf{R}_{ch} \mathbf{R}_{c_{ij}} \mathbf{R}_{ch}^T \leftarrow \mathbf{S} = \mathbf{P} \mathbf{T} \mathbf{P}^{-1}$$

$$\mathbf{x}_{c_j} = \mathbf{R}_{c_{ij}} \mathbf{x}_{c_i'} \leftarrow \mathbf{T} = \mathbf{R}_{c_{ij}}$$

$$\mathbf{x}_{h_j} = \mathbf{R}_{h_{ij}} \mathbf{x}_{h_i'} \leftarrow \mathbf{S} = \mathbf{R}_{h_{ij}}$$

$$\mathbf{x}_{h_i} = \mathbf{R}_{ch} \mathbf{x}_{c_i'} \leftarrow \mathbf{P} = \mathbf{R}_{ch}$$

$$\mathbf{x}_{h_j} = \mathbf{R}_{ch} \mathbf{x}_{c_j}$$

$$\square \text{ skew}(\mathbf{p}_{hij} + \mathbf{p}_{cij}) \mathbf{p}'_{ch} = \mathbf{p}_{hij} - \mathbf{p}_{cij}$$

$$\bullet (\mathbf{p}_{hij} - \mathbf{p}_{cij})^T (\mathbf{p}_{hij} + \mathbf{p}_{cij}) = (\mathbf{R}_{ch} \mathbf{p}_{cij} - \mathbf{p}_{cij})^T (\mathbf{R}_{ch} \mathbf{p}_{cij} + \mathbf{p}_{cij}) = 0$$

• Since $(\mathbf{p}_{hij} - \mathbf{p}_{cij})$ is orthogonal to $\mathbf{p}_{ch}$ and $(\mathbf{p}_{hij} + \mathbf{p}_{cij})$,
 $(\mathbf{p}_{hij} - \mathbf{p}_{cij})$ is colinear with $(\mathbf{p}_{hij} + \mathbf{p}_{cij}) \times \mathbf{p}_{ch}$

$$\bullet |(\mathbf{p}_{hij} + \mathbf{p}_{cij}) \times \mathbf{p}'_{ch}| = |\mathbf{p}_{hij} + \mathbf{p}_{cij}| |\mathbf{p}'_{ch}| \sin \alpha$$

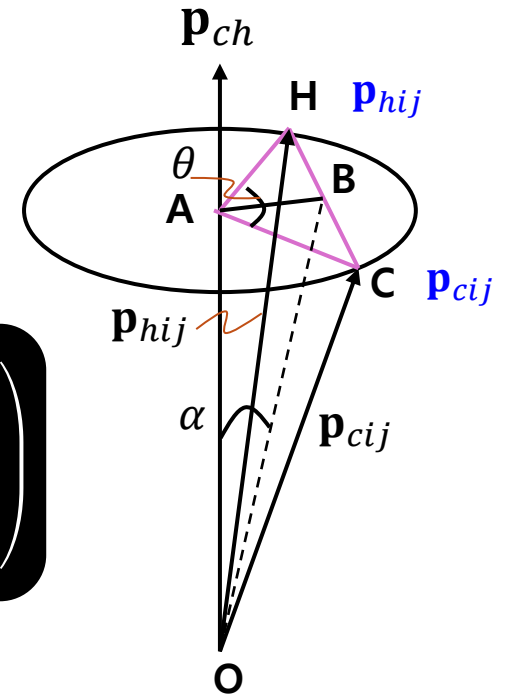
$$= |\mathbf{p}_{hij} + \mathbf{p}_{cij}| \tan \frac{\theta}{2} \sin \alpha$$

$$= 2\overline{OB} \tan \frac{\theta}{2} \sin \alpha$$

$$= 2\overline{AB} \tan \frac{\theta}{2}$$

$$= \overline{CH} = |\mathbf{p}_{hij} - \mathbf{p}_{cij}|$$

$$\begin{pmatrix} \mathbf{p}'_{ch} = \frac{1}{\sqrt{4 - |\mathbf{p}_{ch}|^2}} \mathbf{p}_{ch} \\ \mathbf{p}_{ch} = 2 \sin \frac{\theta}{2} (n_1, n_2, n_3)^T \end{pmatrix}$$



$$\therefore \text{ skew}(\mathbf{p}_{hij} + \mathbf{p}_{cij}) \mathbf{p}'_{ch} = \mathbf{p}_{hij} - \mathbf{p}_{cij}$$

Implementation

```
...
AX = XB for eye-in-hand calibration
# X = T_ch
# A = T_hij (ndarray N X 4 X 4)
# B = T_cij (ndarray N X 4 X 4)
# T_hb: pose of hand frame in base frame (ndarray M X 4 X 4)
# T_oc: pose of world(object) frame in base frame (ndarray M X 4 X 4)
...
return T_ch
```

```
def AXXB_Tsai(T_hb, T_oc):

    # R_ch using the least-square method
    M = T_hb.shape[0]
    P = []; Q = []

    for i in range(M/2):
        j = i + M/2

        H_ij = np.linalg.inv(T_hb[j]) @ T_hb[i]
        C_ij = T_oc[j] @ np.linalg.inv(T_oc[i])

        rH = cv2.Rodrigues(H_ij[0:3,0:3])[0]
        theta = np.linalg.norm(rH)
        rHn = rH / theta
        rH = 2*np.sin(theta/2.0)*rHn #p_h_ij

        rC = cv2.Rodrigues(C_ij[0:3,0:3])[0]
        theta = np.linalg.norm(rC)
        rCn = rC / theta
        rC = 2*np.sin(theta/2.0)*rCn #p_c_ij

        P.append(basic.skew((rHn + rCn).flatten()))
        Q.append(rHn - rCn)

    P = np.array(P).reshape(-1, 3)
    Q = np.array(Q).reshape(-1, 1)
```

skew($\mathbf{p}_{hij} + \mathbf{p}_{cij}$)

$\mathbf{p}_{hij} - \mathbf{p}_{cij}$

$$\begin{pmatrix} P_1 \\ \vdots \\ P_K \end{pmatrix} \mathbf{p}'_{ch} = \begin{pmatrix} Q_1 \\ \vdots \\ Q_K \end{pmatrix}$$

□ Implementation

```
U, W, VT = np.linalg.svd(P)
p_ch = VT.T @ np.linalg.inv(W) @ U.T * Q # p'_ch

theta = 2.0 * np.arctan(np.linalg.norm(p_ch))
p_ch = p_ch / np.linalg.norm(p_ch)
R_ch = np.eye(3) + np.sin(theta)*basic.skew(p_ch) + \
        (1-np.cos(theta))*basic.skew(p_ch)@basic.skew(p_ch)

# t_ch using the least-square method
P = []; Q = []

for i in range(M/2):
    j = i + M/2

    H_ij = np.linalg.inv(T_hb[j]) @ T_hb[i]
    C_ij = T_oc[j] @ np.linalg.inv(T_oc[i])

    P.append(H_ij[0:3,0:3] - np.eye(3))
    Q.append(R_ch @ C_ij[0:3, 3] - H_ij[0:3, 3])

P = np.array(P).reshape(-1, 3)
Q = np.array(Q).reshape(-1,1)

U, W, VT = np.linalg.svd(P)
t_ch = VT.T @ np.linalg.inv(W) @ U.T @ Q # t'_ch

return R_ch, t_ch
```

$$(\mathbf{R}_{hij} - \mathbf{I})\mathbf{t}_{ch} = \mathbf{R}_{ch}\mathbf{t}_{cij} - \mathbf{t}_{hij}$$

Robot-Eye Calibration

- Robot-Eye Calibration
- Tsai & Lenz Method
- **Dornaika & Horaud Method**
- Non-linear Optimization



□ Dornaika & Horaud Method

- Setup for Eye-to-Hand Calibration

From the below figure,

$$\mathbf{T}_o^c \mathbf{T}_h^o = \mathbf{T}_b^c \mathbf{T}_h^b.$$

Letting the knowns

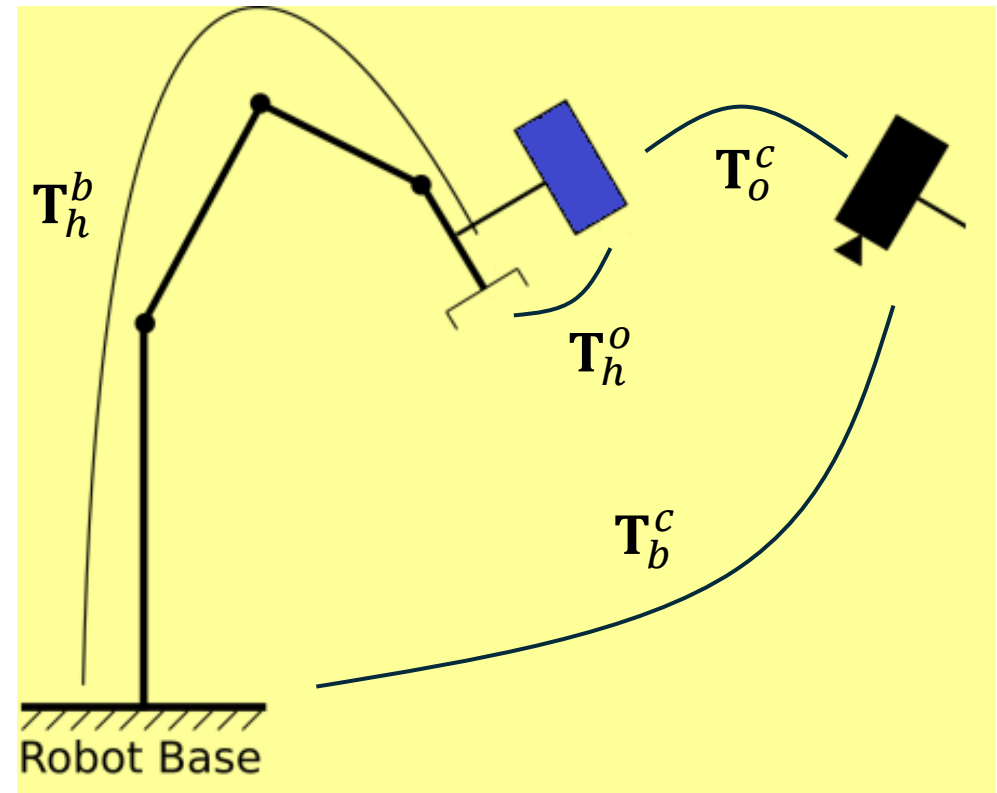
$$\mathbf{T}_o^c = \mathbf{A} \text{ and } \mathbf{T}_h^b = \mathbf{B}, \text{ and}$$

the unknowns

$$\mathbf{T}_h^o = \mathbf{X} \text{ and } \mathbf{T}_b^c = \mathbf{Z},$$

it follows

$$\mathbf{AX} = \mathbf{ZB}.$$



□ Quaternion

- Unit quaternions correspond to the rotation matrices

✓ β rotation around the unit vector $(p_1, p_2, p_3) : \mathbf{q} = \left(\cos \frac{\beta}{2}, p_1 \sin \frac{\beta}{2}, p_2 \sin \frac{\beta}{2}, p_3 \sin \frac{\beta}{2} \right)$

✓ $\mathbf{R} \rightarrow \mathbf{q} = q_0 + iq_x + jq_y + kq_z, (q_0, q_x, q_y, q_z)^T$ where $\|\mathbf{q}\|^2 = q_0^2 + q_x^2 + q_y^2 + q_z^2 = 1$

✓ $\mathbf{q} * \mathbf{r} = (q_0 + iq_x + jq_y + kq_z) (r_0 + ir_x + jr_y + kr_z)$

✓ $\mathbf{q} \cdot \mathbf{r} = q_0 r_0 + q_x r_x + q_y r_y + q_z r_z$

✓ $\bar{\mathbf{q}} = (q_0, -q_x, -q_y, -q_z)^T, \mathbf{q} * \bar{\mathbf{q}} = \bar{\mathbf{q}} * \mathbf{q} = 1$

✓ $\mathbf{R}_A \mathbf{R}_B \rightarrow \mathbf{q}_A * \mathbf{q}_B = Q(\mathbf{q}_A) \mathbf{q}_B = W(\mathbf{q}_B) \mathbf{q}_A$ where

$$Q(\mathbf{q}) = \begin{pmatrix} q_0 & -q_x & -q_y & -q_z \\ q_x & q_0 & -q_z & q_y \\ q_y & q_z & q_0 & -q_x \\ q_z & -q_y & q_x & q_0 \end{pmatrix}, W(\mathbf{q}) = \begin{pmatrix} q_0 & -q_x & -q_y & -q_z \\ q_x & q_0 & q_z & -q_y \\ q_y & -q_z & q_0 & q_x \\ q_z & q_y & -q_x & q_0 \end{pmatrix} \text{ and}$$

$$Q(\mathbf{q})^T Q(\mathbf{q}) = \mathbf{q}^T \mathbf{q} = \mathbf{I}, W(\mathbf{q})^T W(\mathbf{q}) = \mathbf{q}^T \mathbf{q} = \mathbf{I}$$

✓ $\mathbf{R} \mathbf{x} = \mathbf{q} * \mathbf{x} * \bar{\mathbf{q}}$

✓ $\|\mathbf{q} * \mathbf{r}\|^2 = \|\mathbf{q}\|^2 \|\mathbf{r}\|^2$

□ Quaternion

- Implementation

```
...
Rotation -> Quaternion
# rot: rotation matrix (ndarray 3X3)
# return: quaternion (ndarray 4)
...
def quaternion(rot):
    # r_vec: rotation axis vector, ndarray 3X1
    r_vec, _ = cv2.Rodrigues(rot)

    # theta: rotation angle around r_vec
    theta = np.linalg.norm(r_vec)
    if theta < 1e-6:
        # identity matrix 3X3
        return np.array([1.0, 0.0, 0.0, 0.0])

    # unit axis vector
    axis = r_vec / theta

    # calculate the quaternion
    w = np.cos(theta / 2)
    n = axis.flatten() * np.sin(theta / 2)

    return np.array([w, n[0], n[1], n[2]])
```

```
...
Quaternion -> Q matrix
# q: quaternion (ndarray 4)
# return Q matrix (ndarray 4X4)
...
def Qq(q):
    Q = np.zeros((4,4))
    Q[0,:] = np.array([q[0], -q[1], -q[2], -q[3]])
    Q[1,:] = np.array([q[1], q[0], -q[3], q[2]])
    Q[2,:] = np.array([q[2], q[3], q[0], -q[1]])
    Q[3,:] = np.array([q[3], -q[2], q[1], q[0]])

    return Q

...
Quaternion -> W matrix
# q: quaternion (ndarray 4)
# return W matrix (ndarray 4X4)
...
def Wq(q):
    W = np.zeros((4,4))
    W[0,:] = np.array([q[0], -q[1], -q[2], -q[3]])
    W[1,:] = np.array([q[1], q[0], q[3], -q[2]])
    W[2,:] = np.array([q[2], -q[3], q[0], q[1]])
    W[3,:] = np.array([q[3], q[2], -q[1], q[0]])

    return W
```

□ Dornaika & Horaud Method

- From $\mathbf{A}_i \mathbf{X} = \mathbf{Z} \mathbf{B}_i$, $i = 1, \dots, N$

$$\mathbf{R}_{A_i} \mathbf{R}_X = \mathbf{R}_Z \mathbf{R}_{B_i} \text{ ----- } \textcircled{1}$$

$$\mathbf{R}_{A_i} \mathbf{t}_X + \mathbf{t}_{A_i} = \mathbf{R}_Z \mathbf{t}_{B_i} + \mathbf{t}_Z \text{ ----- } \textcircled{2}$$

Representing $\textcircled{1}$ by quaternion gives,

$$\mathbf{q}_{A_i} * \mathbf{q}_X = \mathbf{q}_Z * \mathbf{q}_{B_i}$$

$$Q(\mathbf{q}_{A_i}) \mathbf{q}_X = W(\mathbf{q}_{B_i}) \mathbf{q}_Z .$$

□ Dornaika & Horaud Method

- Now the square norm of the error is defined as:

$$\begin{aligned} & \|Q(\mathbf{q}_{A_i}) \mathbf{q}_X - W(\mathbf{q}_{B_i}) \mathbf{q}_Z\|^2 \\ &= (Q(\mathbf{q}_{A_i}) \mathbf{q}_X - W(\mathbf{q}_{B_i}) \mathbf{q}_Z)^T (Q(\mathbf{q}_{A_i}) \mathbf{q}_X - W(\mathbf{q}_{B_i}) \mathbf{q}_Z) \\ &= \mathbf{q}_X^T Q(\mathbf{q}_{A_i})^T Q(\mathbf{q}_{A_i}) \mathbf{q}_X + \mathbf{q}_Z^T W(\mathbf{q}_{B_i})^T W(\mathbf{q}_{B_i}) \mathbf{q}_Z \\ &\quad - \mathbf{q}_X^T Q(\mathbf{q}_{A_i})^T W(\mathbf{q}_{B_i}) \mathbf{q}_Z - \mathbf{q}_Z^T W(\mathbf{q}_{B_i})^T Q(\mathbf{q}_{A_i}) \mathbf{q}_X \\ &= \mathbf{q}_X^T \mathbf{q}_X + \mathbf{q}_Z^T \mathbf{q}_Z - \mathbf{q}_X^T Q(\mathbf{q}_{A_i})^T W(\mathbf{q}_{B_i}) \mathbf{q}_Z - \mathbf{q}_Z^T W(\mathbf{q}_{B_i})^T Q(\mathbf{q}_{A_i}) \mathbf{q}_X \\ &= \mathbf{v}^T \mathbf{S}_i \mathbf{v} \end{aligned}$$

$$\text{where } \mathbf{v} = \begin{pmatrix} \mathbf{q}_X \\ \mathbf{q}_Z \end{pmatrix}, \quad \mathbf{S}_i = \begin{pmatrix} \mathbf{I} & \mathbf{C}_i \\ \mathbf{C}_i^T & \mathbf{I} \end{pmatrix}, \quad \mathbf{C}_i = -Q(\mathbf{q}_{A_i})^T W(\mathbf{q}_{B_i}).$$

- The total square error norm of entire positions becomes:

$$E(\mathbf{q}_X, \mathbf{q}_Z) = \sum_{i=1}^N \mathbf{v}^T \mathbf{S}_i \mathbf{v} = \mathbf{v}^T \mathbf{S} \mathbf{v} \quad \text{where } \mathbf{S} = \sum_{i=1}^N \mathbf{S}_i = \begin{pmatrix} N\mathbf{I} & \mathbf{C} (= \sum_{i=1}^N \mathbf{C}_i) \\ \mathbf{C}^T & N\mathbf{I} \end{pmatrix}.$$

□ Dornaika & Horaud Method

- To obtain $\mathbf{v}$ to minimize the total square norm subject to $\mathbf{q}_X^T \mathbf{q}_X = 1$, $\mathbf{q}_Z^T \mathbf{q}_Z = 1$, introducing the Lagrange multipliers, it follows that:

$$\begin{aligned}\mathbf{v}^* &= \operatorname{argmin}_{\mathbf{v}} E'(\mathbf{q}_X, \mathbf{q}_Z) = \operatorname{argmin}_{\mathbf{v}} \mathbf{v}^T \mathbf{S} \mathbf{v} + \lambda_1 (1 - \mathbf{q}_X^T \mathbf{q}_X) + \lambda_2 (1 - \mathbf{q}_Z^T \mathbf{q}_Z) \\ &= \operatorname{argmin}_{\mathbf{v}} (N - \lambda_1) \mathbf{q}_X^T \mathbf{q}_X + (N - \lambda_2) \mathbf{q}_Z^T \mathbf{q}_Z + \mathbf{q}_X^T \mathbf{C} \mathbf{q}_Z + \mathbf{q}_Z^T \mathbf{C}^T \mathbf{q}_X + \lambda_1 + \lambda_2. \quad \text{--- (3)}\end{aligned}$$

By differentiating with respect to $\mathbf{q}_X$ and $\mathbf{q}_Z$, we obtain:

$$(N - \lambda_1) \mathbf{q}_X + \mathbf{C} \mathbf{q}_Z = 0,$$

$$(N - \lambda_2) \mathbf{q}_Z + \mathbf{C}^T \mathbf{q}_X = 0$$

From the two equations,

$$\mathbf{q}_X = \frac{-1}{(N - \lambda_1)} \mathbf{C} \mathbf{q}_Z \quad \text{----- (4)}$$

$$\mathbf{C}^T \mathbf{C} \mathbf{q}_Z = (N - \lambda_1)(N - \lambda_2) \mathbf{q}_Z \quad \rightarrow \quad \mathbf{C}^T \mathbf{C} \mathbf{e}_i = \alpha_i \mathbf{e}_i, i = 1, \dots, 4 \quad \text{----- (5)}$$

So, $\mathbf{q}_Z$ is an eigenvector of $\mathbf{C}^T \mathbf{C}$. Substituting (4) and (5) into (3),

$\min(E'(\mathbf{q}_X, \mathbf{q}_Z))$ is obtained at $\nabla E'(\mathbf{q}_X, \mathbf{q}_Z) = 0$ like:

$$\min(E'(\mathbf{q}_X, \mathbf{q}_Z)) = \lambda_1 + \lambda_2$$

□ Eye-to-Hand Calibration

- Since $\mathbf{q}_X^T \mathbf{q}_X = 1$,

$$\begin{aligned}\mathbf{q}_X^T \mathbf{q}_X &= \frac{1}{(N-\lambda_1)^2} \mathbf{q}_Z^T \mathbf{C}^T \mathbf{C} \mathbf{q}_Z = \frac{1}{(N-\lambda_1)^2} \mathbf{q}_Z^T (N - \lambda_1)(N - \lambda_2) \mathbf{q}_Z \\ &= \frac{(N-\lambda_2)}{(N-\lambda_1)} = 1.\end{aligned}$$

$$\therefore \lambda_1 = \lambda_2 = \lambda$$

$$\therefore \min(E'(\mathbf{q}_X, \mathbf{q}_Z)) = \lambda_1 + \lambda_2 = 2\lambda$$

- Now from ⑤,

$$(N - \lambda)^2 = \alpha_i \rightarrow \lambda = N \pm \sqrt{\alpha_i}, \quad i = 1, 2, 3, 4$$

$\therefore$ Select the eigenvalue α_i such that $N \pm \sqrt{\alpha_i}$ is the smallest positive number.

$$\alpha_i \rightarrow \mathbf{e}_i \rightarrow \mathbf{q}_Z \rightarrow \mathbf{q}_X \rightarrow \begin{cases} \mathbf{R}_X \\ \mathbf{R}_Z \end{cases} \rightarrow \mathbf{t}_X, \mathbf{t}_Z (\because \textcircled{2})$$

□ Dornaika & Horaud Method

- From ②,

$$\mathbf{R}_{A_i} \mathbf{t}_X + \mathbf{t}_{A_i} = \mathbf{R}_Z \mathbf{t}_{B_i} + \mathbf{t}_Z$$

$$(\mathbf{R}_{A_i} - \mathbf{I}) \begin{pmatrix} \mathbf{t}_X \\ \mathbf{t}_Z \end{pmatrix} = \mathbf{R}_Z \mathbf{t}_{B_i} - \mathbf{t}_{A_i}.$$

Letting $\mathbf{P}_{3 \times 6}^{(i)} = (\mathbf{R}_{A_i} - \mathbf{I})$, $\mathbf{q}_{3 \times 1}^{(i)} = \mathbf{R}_Z \mathbf{t}_{B_i} - \mathbf{t}_{A_i}$,

$$\mathbf{P} \begin{pmatrix} \mathbf{t}_X \\ \mathbf{t}_Z \end{pmatrix} = \mathbf{B} \text{ where } \mathbf{P} = \begin{pmatrix} \mathbf{P}^{(1)} \\ \vdots \\ \mathbf{P}^{(N)} \end{pmatrix}, \quad \mathbf{Q} = \begin{pmatrix} \mathbf{q}^{(1)} \\ \vdots \\ \mathbf{q}^{(N)} \end{pmatrix}$$

$\therefore \mathbf{t}_X, \mathbf{t}_Z$ can be obtained by the standard least square method

$$\begin{pmatrix} \mathbf{t}_X \\ \mathbf{t}_Z \end{pmatrix}^* = \operatorname{argmin}_{\mathbf{t}_X, \mathbf{t}_Z} \left\| \mathbf{P} \begin{pmatrix} \mathbf{t}_X \\ \mathbf{t}_Z \end{pmatrix} - \mathbf{Q} \right\|^2$$



$$\begin{pmatrix} \mathbf{t}_X \\ \mathbf{t}_Z \end{pmatrix} = \mathbf{V} \mathbf{W}^+ \mathbf{U}^T \mathbf{Q} \text{ where } \mathbf{P} = \mathbf{U} \mathbf{W} \mathbf{V}^T$$

□ Dornaika & Horaud Method

- Implementation

```
'''
AX = ZB for eye-in-hand calibration
# X = T_hc - hand frame in camera frame
# Z = T_bw - base frame in world frame
# A = T_cw (ndarray N X 4 X 4) - camera frame in world frame
# B = T_hb (ndarray N X 4 X 4) - hand frame in base frame
# Feature: feature matrix(ndarray N X 2 X M), N = len(F)
# Model: model-point matrix (ndarray 2 X M)
# return T_hc, T_bw
AX = ZB for eye-to-hand calibration
# X = T_ho
# Z = T_bc
# A = T_oc (ndarray N X 4 X 4)
# B = T_hb (ndarray N X 4 X 4)
# Feature: feature matrix(ndarray N X 2 X M)
# Model: model-point matrix (ndarray 2 X M)
# return T_ho, T_bc

reference: F. Dornaika and R. Horaud, "Simultaneous robot-world and hand-eye calibration,"
IEEE Transactions on Robotics and Automation, vol. 15, no. 5, pp. 851-863, 1999.
'''
```


□ Dornaika & Horaud Method

- Implementation

```
def AXZB_Horaud(A, B):
```

```
    # number of frames
```

```
    N = B.shape[0];
```

```
    C = np.zeros((4,4))
```

```
    # compute C matrix
```

```
    for i in range(N):
```

```
        # q_a <- A[i]
```

```
        q_a = basic.quaternion(A[i][0:3,0:3])
```

```
        Q_a = basic.Qq(q_a)
```

```
        # q_b <- B[i]
```

```
        q_b = basic.quaternion(B[i][0:3,0:3])
```

```
        W_b = basic.Wq(q_b)
```

```
    # C matrix
```

```
    C -= Q_a.T @ W_b
```

```
    # apply SVD to C.T @ C
```

```
    U, W, VT = np.linalg.svd(C.T @ C)
```

$$N \pm \sqrt{\alpha_i}$$

$$C_i = -Q(q_{A_i})^T W(q_{B_i})$$
$$C = \sum_{i=1}^N C_i$$

```
    # q_z corresponding to the largest eigen value
```

```
    dic = {}
```

```
    for i in range(W.shape[0]): # number of eigen values
```

```
        dic[N + np.sqrt(W[i])] = i
```

```
        dic[N - np.sqrt(W[i])] = i
```

```
    lamda = [item for item in sorted(dic.keys()) if item > 0]
```

```
    idx = dic[lamda[0]]
```

```
    q_z = VT[idx,:]
```

```
    # q_x
```

```
    q_x = C @ q_z / (lamda[0] - N)
```

```
    # rotations of X and Z
```

```
    R_X = basic.rotation(q_x)
```

```
    R_Z = basic.rotation(q_z)
```

$$q_x = \frac{-1}{(N - \lambda_1)} C q_z$$

□ Dornaika & Horaud Method

- Implementation

```
# compute t_X, t_Z by least square method (P t = Q)
P = np.zeros((3*N,6))
Q = np.zeros((3*N,1))

for i in range(N):
    P[3*i:3*i+3, 0:3] = A[i][0:3,0:3]; P[3*i:3*i+3, 3:6] = -np.identity(3)
    Q[3*i:3*i+3,0] = R_Z @ B[i][0:3, 3] - A[i][0:3, 3]

# apply SVD to P
assert P.shape[0] >= P.shape[1], "number of equations should be larger than
U, W, VT = np.linalg.svd(P)

# pseudo inverse of W
Winv = np.linalg.inv(np.diag(W))
Winv = np.hstack((Winv, np.zeros((P.shape[1], P.shape[0]-P.shape[1]))))

t = VT.T @ Winv @ U.T @ Q
t_X = t[0:3,0]
t_Z = t[3:6,0]

#homogeneous transformation
X = np.identity(4); Z = np.identity(4)
X[0:3, 0:3] = R_X; Z[0:3,0:3] = R_Z
X[0:3, 3] = t_X; Z[0:3, 3] = t_Z

return X, Z
```

$$\begin{pmatrix} t_X \\ t_Z \end{pmatrix} = VW^+U^TQ$$

Robot-Eye Calibration

- Robot-Eye Calibration
- Tsai & Lenz Method
- Dornaika & Horaud Method
- **Non-linear Optimization**



□ Non-linear Optimization in $AX=XB$

- Cost Function

- ✓ Frobenius norm of a matrix $\mathbf{M}_{m \times m}$

$$\|\mathbf{M}\|_F^2 = \text{trace}(\mathbf{M}^T \mathbf{M}) = \sum_{i=1}^m \lambda_i^2 \text{ where } \lambda_i \text{ is an eigenvalue of } \mathbf{M}_{m \times m}$$

- ✓ $C(X(\alpha, \beta, \gamma, t_{X_x}, t_{X_y}, t_{X_z})) = \|\mathbf{A}\mathbf{X} - \mathbf{X}\mathbf{B}\|_F^2$

$$\text{where } \mathbf{A} = \begin{pmatrix} \mathbf{A}_1 \\ \vdots \\ \mathbf{A}_N \end{pmatrix}, \mathbf{B} = \begin{pmatrix} \mathbf{B}_1 \\ \vdots \\ \mathbf{B}_N \end{pmatrix}.$$

□ Non-linear Optimization in $AX=ZB$

- Cost Function

- ✓ Frobenius norm of a matrix $\mathbf{M}_{m \times m}$

$$\|\mathbf{M}\|_F^2 = \text{trace}(\mathbf{M}^T \mathbf{M}) = \sum_{i=1}^m \lambda_i^2 \text{ where } \lambda_i \text{ is an eigenvalue of } \mathbf{M}_{m \times m}$$

- ✓ $C\left(X(\alpha, \beta, \gamma, t_{X_x}, t_{X_y}, t_{X_z}), Z(a, b, c, t_{Z_x}, t_{Z_y}, t_{Z_z})\right) = \|\mathbf{AX} - \mathbf{ZB}\|_F^2$

$$\text{where } \mathbf{A} = \begin{pmatrix} \mathbf{A}_1 \\ \vdots \\ \mathbf{A}_N \end{pmatrix}, \mathbf{B} = \begin{pmatrix} \mathbf{B}_1 \\ \vdots \\ \mathbf{B}_N \end{pmatrix}.$$

□ Non-linear Optimization in $AX=ZB$

- Eye-in-Hand

- ✓ From the right figure

$$\mathbf{T}_c^w \mathbf{T}_h^c = \mathbf{T}_b^w \mathbf{T}_h^b \Leftrightarrow \mathbf{A} \mathbf{X}_h^c = \mathbf{Z}_b^w \mathbf{B}$$

$$\mathbf{T}_w^c = \mathbf{T}_h^c (\mathbf{T}_b^w \mathbf{T}_h^b)^{-1} = \mathbf{X}(\mathbf{ZB})^{-1}$$

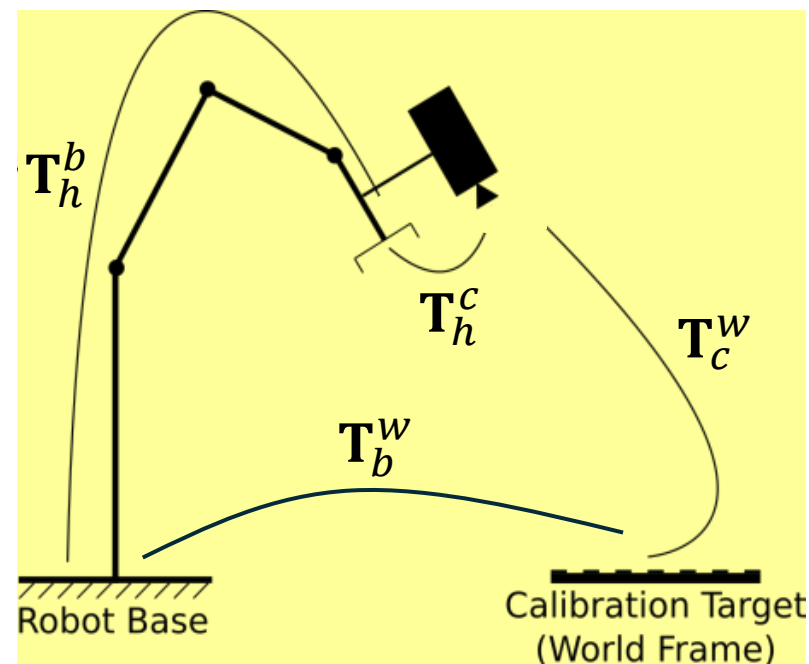
- ✓ Image projection

$$s \left(\begin{pmatrix} \hat{\mathbf{u}}_{ij} \\ 1 \end{pmatrix} \right) = f_{\mathbf{K}, \mathbf{k}} \left(\mathbf{T}_w^{c_i} \begin{pmatrix} \mathbf{w}_j \\ 1 \end{pmatrix} \right) = f_{\mathbf{K}, \mathbf{k}} \left(\mathbf{X}(\mathbf{ZB}_i)^{-1} \begin{pmatrix} \mathbf{w}_j \\ 1 \end{pmatrix} \right)$$

where i is a frame index, j is a feature index

- ✓ Image projection error = Cost

$$C \left(X(\alpha, \beta, \gamma, t_{x_x}, t_{x_y}, t_{x_z}), Z(a, b, c, t_{z_x}, t_{z_y}, t_{z_z}) \right) = \sum_{i=1}^N \sum_{j=1}^M \|\mathbf{u}_{ij} - \hat{\mathbf{u}}_{ij}\|^2$$



□ Non-linear Optimization in $AX=ZB$

- Eye-to-Hand

- ✓ From the right figure

$$\mathbf{T}_o^c \mathbf{T}_h^o = \mathbf{T}_b^c \mathbf{T}_h^b \Leftrightarrow \mathbf{A} \mathbf{X}_h^o = \mathbf{Z}_b^c \mathbf{B}$$

$$\mathbf{T}_o^c = \mathbf{T}_b^c \mathbf{T}_h^b \mathbf{T}_h^o = \mathbf{ZB}(\mathbf{X})^{-1}.$$

For $i = 1, \dots, N$,

$$\therefore \mathbf{T}_{o_i}^c = \mathbf{T}_b^c \mathbf{T}_{h_i}^b \mathbf{T}_h^o = \mathbf{ZB}_i(\mathbf{X})^{-1}$$

- ✓ Image projection

$$s \left(\begin{pmatrix} \hat{\mathbf{u}}_{ij} \\ 1 \end{pmatrix} \right) = f_{K,k} \left(\mathbf{T}_{o_i}^c \begin{pmatrix} \mathbf{o}_j \\ 1 \end{pmatrix} \right) = f_{K,k} \left(\mathbf{ZB}_i(\mathbf{X})^{-1} \begin{pmatrix} \mathbf{o}_j \\ 1 \end{pmatrix} \right)$$

where i is a frame index, j is a feature index

- ✓ Image projection error = Cost

$$C \left(X(\alpha, \beta, \gamma, t_{X_x}, t_{X_y}, t_{X_z}), Z(a, b, c, t_{Z_x}, t_{Z_y}, t_{Z_z}) \right) = \sum_{i=1}^N \sum_{j=1}^M \|\mathbf{u}_{ij} - \hat{\mathbf{u}}_{ij}\|^2$$

